

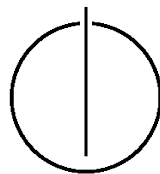
FAKULTÄT FÜR INFORMATIK

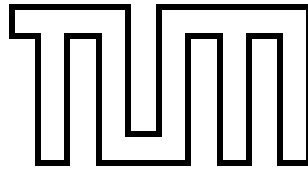
DER TECHNISCHEN UNIVERSITÄT MÜNCHEN

Masterarbeit in Informatik

**Monkey: Automated debugging of deployed
distributed systems**

Safey Abdel Halim





FAKULTÄT FÜR INFORMATIK

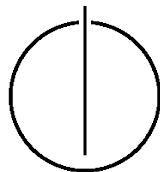
DER TECHNISCHEN UNIVERSITÄT MÜNCHEN

Masterarbeit in Informatik

Monkey: Automated debugging of deployed distributed
systems

Monkey: Automatisches Debugging von verteilten
Systemen

Author: Safey Abdel Halim
Supervisor: Christian Grothoff, Ph.D.
Advisor: Christian Grothoff, Ph.D.
Date: July 18, 2012



Ich versichere, dass ich diese Masterarbeit selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

München, den 18. Juli 2012

Safey Abdel Halim

Acknowledgments

The project presented in this thesis was essentially a part of `gnunet` [11]. Many `gnunet` utilities and data structures were used. They provided a great help, and saved a lot of development time.

The bug characterization and the expressions value analysis presented in Chapter 5 were realized by using `gdb` [25] and `valgrind` [22]. The library `libmigd` [19] was very useful in facilitating the automatic control of `gdb`.

I would like to thank my advisor Christian Grothoff for his help in designing, and developing this work. For me, his explanations were crucial to understand many concepts, and his ideas were key to solve many problems. I also thank him for his valuable and patient rounds of reviews and advice on the writing.

I also would like to thank the `gnunet` team at the Technical University of Munich, especially Matthias Wachs, and Nathan Evans who were always glad to help me and answer my questions.

Abstract

Debugging is tedious and time consuming work that, for certain types of bugs, can and should be automated. Debugging distributed systems is more complex due to time dependencies between interacting processes. Another problem is duplicate bug reports in bug repositories. Finding bug duplicates is hard and wastes developers' time, which may affect the development team's rate of bug fixes and new releases.

In this master thesis, we will present Monkey: a new tool that provides a solution for automating, investigating, and characterizing bugs. Monkey is fully automated and does not incorporate any manual intervention but for triggering the tool. Monkey uses existing debugging tools, and select between at them at runtime based on the bug category. It generates a bug report that contains a detailed description of the bug as well as the values of relevant expressions. It also supports backtracking which permits showing the program states at many points of the program's execution history.

Monkey provides a solution for the bug report duplicates problem. The solution is based on comparing the bug characteristics available in the Monkey bug reports, instead of comparing human textual descriptions in traditional bug reports.

Monkey is particularly suitable for distributed systems due to its autonomy. We will present Monkey's key design goals and architecture and give runtime examples demonstrating the viability of our approach.

Contents

Acknowledgements	vii
Abstract	ix
Outline of the Thesis	xiii
I. Introduction and Theory	1
1. Introduction	3
2. Background Theory	7
2.1. Distributed Systems	7
2.2. Debugging	8
2.2.1. Bug Classification	9
2.2.2. Bug Characterization	9
2.2.3. Bug Uniqueness	10
2.3. Slicing	11
2.4. Backtracking	11
3. Approach	13
II. System Architecture and Interaction	17
4. System Architecture	19
4.1. Static analysis (The Monkey Seaspider)	19
4.2. Postmortem analysis (The Monkey pathologist)	19
4.3. Bug report analysis (The Monkey entomologist)	22
5. Implementation	23
5.1. Seaspider	23
5.2. Monkey Expression Database (EDB):	25
5.3. Monkey Pathologist	27
5.3.1. Strategy	27
5.3.2. Scope Depth	30
5.3.3. Backtracking	32
5.3.4. Monkey Pathologist's Components	35
5.4. Monkey Entomologist	36

III. Results and Conclusion	41
6. Runtime Examples	43
6.1. Demonstration Overview	43
6.1.1. Example: NULL pointer dereference	43
7. Discussion and Future Work	51
7.1. Presentation	51
7.2. Monkey Pathologist	51
7.3. Monkey Seaspider	52
7.4. Monkey Entomologist	52
7.5. Conclusion	52
Appendix	55
A. Appendix	55
A.1. Downloading and installing Monkey	55
A.2. Monkey pathologist's command arguments	55
A.3. Entomologist's configuration	56
A.4. Bug reports	57
A.4.1. Division by zero bug with backtracking enabled	57
Bibliography	63

Outline of the Thesis

Part I: Introduction and Theory

CHAPTER 1: INTRODUCTION

This chapter presents an introduction to debugging problems and the motivation to the solution presented in this thesis.

CHAPTER 2: BACKGROUND THEORY

This chapter provides an idea about the background theory that was employed for designing and implementing the tool described by this thesis.

CHAPTER 3: APPROACH

This chapter serves as the entry point to describing the tool. It provides a general idea about how the system works to solve the problem in question.

Part II: System Architecture and Interaction

CHAPTER 4: SYSTEM ARCHITECTURE

This chapter provides an explanation of the system architecture, the role of each of the system components, and how those roles are related to the approach presented in Chapter 3.

CHAPTER 5: IMPLEMENTATION

This chapter describes the implementation of each of the system components.

Part III: Results and Conclusion

CHAPTER 7: RUNTIME EXAMPLES

This chapter presents results of running the tool on some examples, and shows the viability of our solution.

CHAPTER 8: DISCUSSION AND FUTURE WORK

This chapter discusses pros and cons of the system. What features it supports, and what features are not supported. It discusses the scope of the future work that can be done in order to improve the system, and how it can be extended to address new problems.

Part I.

Introduction and Theory

1. Introduction

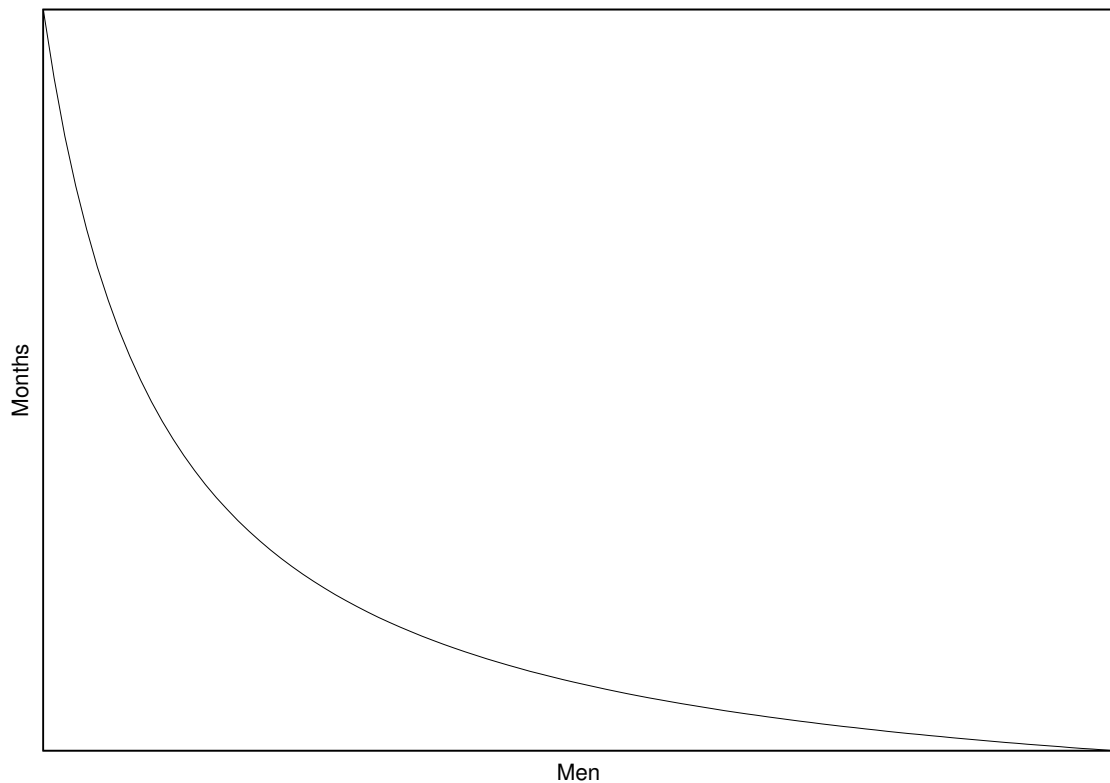


Figure 1.1.: Time versus number of workers in a perfectly partitionable task.

While programming may be an art, debugging is often a rather dull and deterministic process, consisting of the repeated application of well-known steps to narrow down the source of a problem.

Debugging might represent a roadblock in the software life cycle because it is a sequential task that cannot be subdivided between different team members. In software projects, the unit used in estimating and scheduling tasks is the man-month [6]. The man-month unit is the number of men allocated in order to finish a task in one month. In his essay entitled: *The Mythical Man-Month*, Brooks argues: "Men and months are interchangeable commodities only when a task can be partitioned among many workers with no communication among them" [6]. Figure 1.1 shows that adding more men to a perfectly partitionable task will reduce the amount of time necessary for task completion. For debugging, however, adding more men does not affect the task completion time, as debugging is sequential, and there-



Figure 1.2.: Debugging is a sequential task. Number of months is the same no matter how much man power we add.

fore cannot be partitioned. This is shown in Figure 1.2.

In case of bugs that result in misbehavior that is easily classified as wrong, such as crashing the application or violating an assertion, developers always essentially trace back the control and data flow that lead to the safety or correctness violation. This technique is also known as slicing [3]. Depending on the bug, slicing programs manually with a debugger can be trivial or exhaustingly tedious, whereas the final change to the application is often relatively simple. This is particularly true for distributed applications where problems may only arise once many components of the system begin to interact and attaching a debugger to the right process at the right time is far from trivial. Automating debugging is particularly urgent in this case because delays introduced by manual debugging can fundamentally alter timing-dependent behavior of the distributed system.

Existing tools roughly fall into two categories. Offline tools enable the collection of crash information on end-user systems. Such crash handlers are integrated in many operating systems, one modern example being Appport for Ubuntu [27]. The information collected by these tools is typically limited to values of registers, call stacks or even a complete memory dump of the affected process. Collecting and sending complete memory dumps may represent a privacy violation, as memory dumps may contain personal information like the users' public key. Another problem is accessibility. For some distributed systems

like `tor` [4], or `gnunet` [11], developers don't usually have access to the nodes where a bug is discovered. The information collected by traditional offline tools might not be sufficient for the developer to reproduce, and therefore fix the bug. And of course they cannot access and debug the client nodes.

Less common are online tools, which provide developers with an enhanced environment for interactive debugging. Such tools can help trace data flows [2] or even allow the developer to run the program backwards [3]. Online tools require frequent interaction with the developer and are thus often not suitable for distributed systems or deployment on end-user systems.

In this thesis, we present Monkey, a new tool for automated debugging that is supposed to help developers with the tedious work of characterizing bugs and finding all of the information necessary to identify the source of a problem. Solving the bug itself is left entirely to the human developer; Monkey is only supposed to automate the monkey work - the basic, scriptable steps decent programmers could (manually) perform in their sleep to gather information about a bug. Ideally, the information collected by Monkey will be sufficient for the developer to immediately resolve the problem, but in any case it should provide a good starting point for a manual investigation.

Generating more bug reports is a double-edged sword — often development teams have a large number of reports in their bug database and insufficient time to inspect them all. Automatically reporting bugs thus creates the danger of drowning developers in a flood of duplicate reports. Monkey addresses this problem by including tool-support to detect duplicates; the uniform bug reporting format facilitates this by avoiding variations introduced by human reporters.

In the remainder of this thesis we will present Monkey's architecture and key features, present some experimental results demonstrating the current capabilities of the implementation and discuss and the plans for future work.

2. Background Theory

2.1. Distributed Systems

A distributed information system is defined as the “*combination of information processing facilities, data communication facilities, and endpoint facilities. Together, these support the movement and processing of files, programs, data, messages, and transactions*” [7]. Grid computing and parallel computing systems are typical examples of distributed information systems. While traditional computing systems are concerned with process allocation and resource management on a single machine, distributed systems are concerned with those problems in a hybrid environment: the resources are not necessarily running on the same machine, but may be on different machines connected via a network. This introduces complexities in designing, implementing, and debugging such systems.

Parallel programming is defined as “*the programming multiple computers, or computers with multiple internal processors, to solve a problem at a greater computational speed than with a single computer*” [17]. Debugging a parallel program is of great complexity. A parallel program can be composed of multiple threads, or multiple communicating services, those threads or services might be sharing common resources. Attaching a manual debugger to one of the threads or the services will make it significantly slower, and may alter the overall system behavior. Specialized debugging tools have been developed for parallel programming environments [18].

Replay debugger [18] is a tool designed for multi-threaded embedded software. It’s a record/replay framework that runs as a layer between multithreaded applications and the operating system’s kernel. During record phase, it intercepts the inter-process communication and the device IO function calls, and logs the returned error codes and data before returning to the applications. Therefore it can reconstruct the happened-before relationships among the execution of the multiple threads and the IO events. It constructs a partial order of threads interactions. During the replay phase the concurrent execution can be repeated, and the threads can be controlled individually using the constructed partial order. During the replay phase, the developer can observe the program’s behavior without any runtime impact.

While the replay debugger provides a solution for the time dependency problem that often occurs when debugging parallel programs, it is still a manual tool: the developer has to manually analyze the code during the replay phase. Analyzing program expressions, and bug characterization are tedious and time consuming tasks, but they can be subject to automation. The debugging solution proposed in this thesis attempts to automate the expressions’ analysis and the bug characterization, and to provide developers with a bug report that can directly be filed to a bug tracking tool without any human intervention. Therefore Monkey is a fully automated debugging solution.

2.2. Debugging

The entry point for addressing how to automate the debugging process is to understand it. That is, to understand how developers do debugging. Although debugging might seem personal in the sense that the developer's ability to identify and solve the problem based on his background knowledge and his technical skills, it also has its common aspects: it's a set of logical operations whose aim is to identify a set of program expressions contributing to the problem (the set of relevant expressions). Usually, the developer resumes the execution when the program control is in a non-interesting area, then the developer pauses the program when control approaches the location at which the error occurred or the program crashed. The developer then starts examining the values of the sub-expressions of that expression at which the problem occurred. He monitors the call-trace and the conditions that led to this particular expression.

The debugging process has three inputs and one output. The three inputs are:

- The source code.
- The program input.
- The error report which is the reason for debugging.

Of course this the error report can be replaced by a wrong or unexpected behavior of the program (a logic error). The output of the debugging process is:

- The identification of the problem that caused the error.

If the program is fairly simple, the developer starts it in debugging mode by feeding the inputs, and move forward while observing the values of the relevant expressions, until reaching the erroneous expression. However, for many large programs, such an analysis is too difficult to perform. One logical way to proceed in such situation is to *think backwards* [3]

Agrawal proposed in [3] a way to break down an analysis for an erroneous program into the following set of tasks:

1. Problem occurred while executing a certain expression.
2. Determine which statements in the code had an influence on the value of a given variable observed at a given location.
3. Select one of these statements at which to examine the program state.
4. Recreate the program state at the selected statement to examine specific variables.

Determining the statements of the code influencing the value of a certain variable is referred to as *slicing*. Recreating the program state at the selected statement to examine specific variables is referred to as *backtracking*.

2.2.1. Bug Classification

The term “bug” is used to describe a hardware or a software malfunction. The following is the IEEE standard classification for software anomalies [1]:

- **Defect:** An imperfection or deficiency in a work product where that work product does not meet its requirements or specifications and needs to be either repaired or replaced.
- **Error:** A human action that produces an incorrect result.
- **Failure:** (A) Termination of the ability of a product to perform a required function or its inability to perform within previously specified limits. (B) An event in which a system or system component does not perform a required function within specified limits. A failure may be produced when a fault is encountered
- **Fault:** A manifestation of an error in software.
- **Problem:** (A) Difficulty or uncertainty experienced by one or more persons, resulting from an unsatisfactory encounter with a system in use. (B) A negative situation to overcome.

The previous classes are of course related, however, we are particularly interested in automating the discovery and reporting of *failures*, and *faults*.

2.2.2. Bug Characterization

Bug classification distinguishes between software bugs according to their severity. A typical bug classification can be employed in bug tracking systems for example, which will help software development teams to prioritize which bugs must be solved first according to how critical they are to the project or the the customer. The classification, however, does not describe the different types of bugs. Such a description is provided by bug characterization. A typical characterization of software bugs is provided by [26]. Although the study was based on an analysis of two large deployed IBM [13] database management systems: DB2 [14], and IMS [15], the discussed characterization can be applied to other software applications. According to [26], the characterization is as follows:

- **Allocation Management:** One module deallocates a region of memory before it has completely finished using the region. After the region is reallocated, the original module continues to use it in its original capacity.
- **Copying Overrun:** The program copies bytes past the end of a buffer.
- **Data Error:** An arithmetic miscalculation or other error in the code makes it produce or read the wrong data.
- **Interface Error:** A module’s interface is defined incorrectly or used incorrectly by a client.
- **Memory Leak:** The program does not deallocate memory it has allocated.

- **Pointer Management:** A variable containing the address of data was corrupted. For example, a linked list is terminated by setting the last chain pointer to NIL when it should have been set to the head element in the list.
- **Statement Logic:** Statements were executed in the wrong order or were omitted. For example, a routine returns too early under some circumstances. Forgetting to check a routine's return code is also a statement logic error.
- **Synchronization:** An error occurred in locking code or synchronization between threads of control.
- **Uninitialized Variable:** A variable containing either a pointer or data is used before it is initialized.
- **Undefined State:** The system goes into a state that the designers had not anticipated. For example, the program may have no code to handle an end-of-session message which arrives before the session is completely initialized.
- **Unknown:** The error report described the effects of the error, but not adequately enough for us to classify it.
- **Wrong Algorithm:** The program works, but uses the wrong algorithm to do the task at hand. Usually these were performance related problems.
- **Other:** We understood what the error was, but could not fit it into a large enough category.

The solution proposed in this thesis characterizes bugs of the types: pointer management, statement logic, allocation management, and division by zero which is not directly mentioned.

2.2.3. Bug Uniqueness

Redundancy of bug reports is a common problem among software development teams. It's highly probable that many users encounter the same problem, and therefore file the same bug report. Problems may arise from having duplicate bug reports in bug tracking systems. Manual bug triaging will waste developers time and may have negative implications on software quality and release dates.

Prior research tried to tackle this particular problem of bug report redundancy, or inversely, trying to figure out a way to ensure bug report uniqueness. In [20], Nagwani, and Singh proposed an approach for detecting bug duplicates. They distinguish between a *bug duplication*, and *bug similarity*. According to the definition in [20], two bugs are similar if both of them are fixed by the same implementation, while two bugs are said to be duplicates, if the same bug is reported many times using different words. In this thesis, we are interested in *bug duplication*. In [20], the approach of detecting either duplication or similarity is based on the bug's *textual* input: the summary, the description, and the comments. Therefore, a text data mining approach is employed, and which is not particularly related to the bug characteristics: the expression values, and the stack-trace.

Similarly, the idea of work done by Hiew in [12] is to build a model of reports in a bug repository. This model groups similar reports into *centroids*. A *centroid* corresponds either to a group of duplicate bug reports, or a single report with no duplicates. When a new report arrives, the report is compared to each centroid, looking for occurrences of high similarity; reports from highly similar groups are considered potential duplicates. However, building the *centroids*, and comparing new reports is again based on the textual values of the report's summary, and description fields.

In this thesis, we suggest a new approach for insuring bug uniqueness. Comparing bugs is not dependent anymore on text entered by a programmer for describing the problem. Although this technique has been proven useful, it may have the disadvantage of being strongly dependent on a human component that cannot be controlled. Instead, Monkey decides on whether bug reports are similar or not based on the bugs characteristics. Monkey's output is a report containing a detailed description of a bug like the values of expressions related to the problem. Comparison is done by checking two reports, and awarding similarities in these characteristics with weighted scores.

2.3. Slicing

The term slicing means determining the set of variables that influence the value of a certain variable in a program. The concept of program slicing was introduced for the first time in [28]. *"But the original notion of a program slice was that of a static slice which is obtained irrespective of the values of the input variables"* [3]. A static slice is the set of variables in a program P that influence the value of a certain variable X. Static slicing is calculated by examining the program's data and control without actually running the program. Dynamic slicing, on the other hand, means that the set of variables influencing a certain program variable are determined at program runtime. A debugging tool using dynamic slicing has to record the program's execution to determine the runtime influence set on variables, then a programmer debugs the recorded program with the assistance of the calculate influence sets.

The solution proposed in this thesis employs a different slicing technique, we call it post-mortem slicing. In post-mortem slicing, the target program is run normally until it crashes, after that the values of relevant expressions to the failure are examined. Determining the set of relevant expressions is done with the assistance of a database containing all the target program's expressions. Post-mortem slicing differs from the normal dynamic slicing because the analysis starts only after the program crashes (post-mortem), and that it does not necessarily need to record the program's execution before debugging.

2.4. Backtracking

Execution backtracking means running a program in the reverse order. Backtracking is a useful feature usually offered by modern debugging tools that helps developers to understand the behavior of faulty programs. In the context of manual debugging, a developer usually sets a break point at (or near) the location in the code where his program failed. He runs the program normally until reaching that (interesting) location, then starts examining expression values. Often, the developer needs to run steps of the program in the reverse

order to examine, for example, how the value of a certain variable changes with program execution.

To support execution backtracking, a debugging tool has to have the capability of recording the execution path of a program. SPYDER, a debugging tool described in [3], supports execution backtracking. As SPYDER is a dynamic slicing debuggign tool, it normally records program execution history: in dynamic slicing, control flow, and data flow are recorded at runtime to determine the influence variables. Upon stepping back over a program's statement, SPYDER restores the previously saved values of variables modified by that statement. Another tool that supports execution backtracking is `gdb` [25], it enables a program to be recorded during its forward execution, and then replays it backwards either continuously, or step by step. In the next chapter, we will explain our approach to support execution backtracking.

3. Approach

The tool proposed in this thesis, Monkey, is an offline tool, suitable for use on end-user systems. We're particularly interested in debugging large-scale distributed systems consisting of tens of thousands or millions of processes running on a similar number of machines. As a result, manual interaction of the developer with each installation is not feasible. Instead, Monkey must autonomously collect information about issues (possibly in soft real-time to not disrupt interactions with other components of the distributed system) and report the resulting data to a central bug database. Monkey's autonomy sets it apart from existent debugging technologies in which developers manually participate in collecting information and characterizing bugs. That makes Monkey's approach unique.

In addition to capturing information about the bug, key features of Monkey include bug characterization (which type of bug was encountered) and taking appropriate follow-up actions (such as adding watch points during re-execution for slicing, or using alternative instrumentations to collect different information).

Finally, Monkey also attempts to determine which faults actually have a different cause; the goal is to avoid filing the same bug many times. This is important to avoid wasting developer's time on the investigation of many redundant bug reports. Monkey's bug report uniqueness approach is also autonomous: it does not compare bug reports based on any user effort like writing a textual description for the bug. Instead, it depends on comparing bug characteristics collected during program analysis, and automatically written to the bug report. The bug characteristics include file name, function name, and line number, and also the values of relevant expressions, sub-expressions, stack trace, and execution history.

Monkey is composed of several components; the component responsible for automated debugging is written in C. Currently, it only analyzes C programs.

When a user runs a buggy program with Monkey, Monkey automatically debugs the program and generates a report that contains the following information:

- *Bug Characterization*: A description of the bug category, with information about the bug's location (file name, function name, line number). The current implementation of Monkey distinguishes between four categories of bugs:
 1. NULL Pointer Exception
 2. Bad Memory Access
 3. Assertion Failure
 4. Division By Zero
- *Expressions*: A listing of the values of all sub-expressions of the faulty expression, and a set of expressions residing in the same scope.

3. Approach

- *Stacktrace*: The listing of relevant expressions in the call trace from the program entry point to the point at which the program crashed.
- *Reverse Execution History*: When backtracking is enabled the expression values are provided at several points in the program's execution history.

Our approach is based on debugging programs using existing debugging tools. In other words, it's the developer (the code monkey) who is replaced and not the debugger. For bug characterization, Monkey uses two debugging tools: `gdb` [25], and `valgrind` [22]. Essentially, Monkey runs the program with `gdb` until it crashes, then it queries `gdb` for failure location, and the signal received. It starts post-mortem analysis by examining the values of the sub-expressions of the one that failed. Using the failure information collected from `gdb`, and the values of the relevant expressions, Monkey categorizes the bug. If the previous analysis is inconclusive, Monkey re-runs the program with `valgrind` for memory check. Finally, Monkey writes its findings: bug characterization, expression values, and `valgrind` report (if available) to its bug report.

Currently, Monkey generates its bug report in XML format, and it can submit it in two ways: it can save the report on disk, or send it via email to a predefined recipient. XML is used to facilitate the bug report comparison.

Similarity between bugs is measured by comparing their XML reports. Monkey compares the bug characteristics of two bug reports, and generates a score (a percentage) that determines how similar the two bugs are. The higher the similarity score gets, the higher the potential of the two bugs of being duplicates. Each of the characteristics has a configurable weight, the weight determines its importance and therefore its contribution to the similarity score.

To conclude, our first goal is to automatically characterize bugs using existing debugging tools and to provide the developer with a conclusive bug report that contains the failure reasons and the values of the contributing expressions to the failure. The second goal is that the content of those bug reports should facilitate bug comparison, and therefore prevent bug duplicates.

- **Bug characterization**: As our post-mortem slicing approach is based on automatically using `gdb`. One problem is how to launch it and programmatically control it. `gdb` provides a machine interface that can be used by communicating frontends. We used `libmigd` [19], a C library that wraps `gdb`'s machine interface. Another challenge is: given the failure location, and the signal received, how to infer the set of relevant expressions to the failure. Our approach is to build a database that contains all the target program's expressions, and their locations. Using the failure line number, we retrieve the subexpressions, and those expressions residing in the same scope from the database at runtime. `gdb` is again automatically prompted for the values of the relevant expressions. We designed a static analysis module that parses the target program's source tree and builds the expression database. Another issue is how to support backtracking. For enabling backtracking `gdb` is used to record the target program's execution until the crash. Afterwards, the previously mentioned expression value analysis is repeated at each point in the program's execution history. Another challenge is when to decide to launch `valgrind`. A typical case is that of a SIGSEGV

where `gdb` assisted value analysis didn't show any expression set to `NULL`. In that case, `valgrind` is launched for memory inspection.

- ***Bug reporting and uniqueness:*** Having automatically generated bug reports that contain information about bug location, category, and a set of relevant expressions and their values is very useful to objectively comparing bugs. One complication is how to use these information. The use of XML in generating the bug reports facilitates reports' parsing. Our approach is to assign a configurable weight to each of the bug characteristics. For example, there is a weight for the bug category, the line number, and the file name. There are also weights for the stack trace including information about functions, expressions, and their values. The final similarity score is the sum of all those weighted characteristics. Configurable weights are key to fine tune our bug comparison module to achieve better bug triaging results.

Part II.

System Architecture and Interaction

4. System Architecture

In this chapter, we will discuss Monkey's architecture. Different system components and the relations between those components will be explained.

Figure 4.1 shows Monkey's system architecture. Monkey is composed of three main components: A static analysis component, a postmortem analysis component, and a bug report analysis component.

4.1. Static analysis (The Monkey Seaspider)

In Chapter 3, we mentioned that post-mortem analysis requires a database that contains all of the target program's expressions. During the debug process, Monkey uses this database to retrieve the relevant expressions to the one that caused the program's failure. Monkey cannot build that database at runtime (while debugging a program): the target program's source code might not be available at runtime, and building the expression database has its time and space requirements that might negatively affect the debugging speed. Therefore, the expression database must be built offline (before debugging the target program). Building the expression database requires a static analysis of the target program's source code. Currently, static analysis is only limited to program's expressions. Future work, however, may incorporate a more sophisticated static analysis that includes data-flow analysis or inter-procedural analysis.

Monkey's static analysis component is called Seaspider. Its role is to parse the target program's source tree, and build the Monkey expression database, which is a database that should contain all the target program's expressions. Seaspider's input is the path to the target program's source tree. Its output is the expression database file. Seaspider recursively parses the code and extracts C language expressions defined by the C grammar, and inserts them into the expression database.

The Monkey expression database is a simple `sqlite` database [24] that contains only one table called "Expression". Seaspider inserts a record in table "Expression" for each expression it encounters while parsing the target program's source tree. The expression record specifies the expression syntax, the source file name where it resides, the start line number of the expression, the scope end line number of the expression, and a flag specifying if the expression represents a function call.

4.2. Postmortem analysis (The Monkey pathologist)

The automated debugging work is done by the postmortem analysis component, or Monkey pathologist. The pathologist is a C program that takes a path to a binary file (the program to debug), and the path to its corresponding expression database. The pathologist runs the target program and waits until it crashes. When the program crashes, it starts

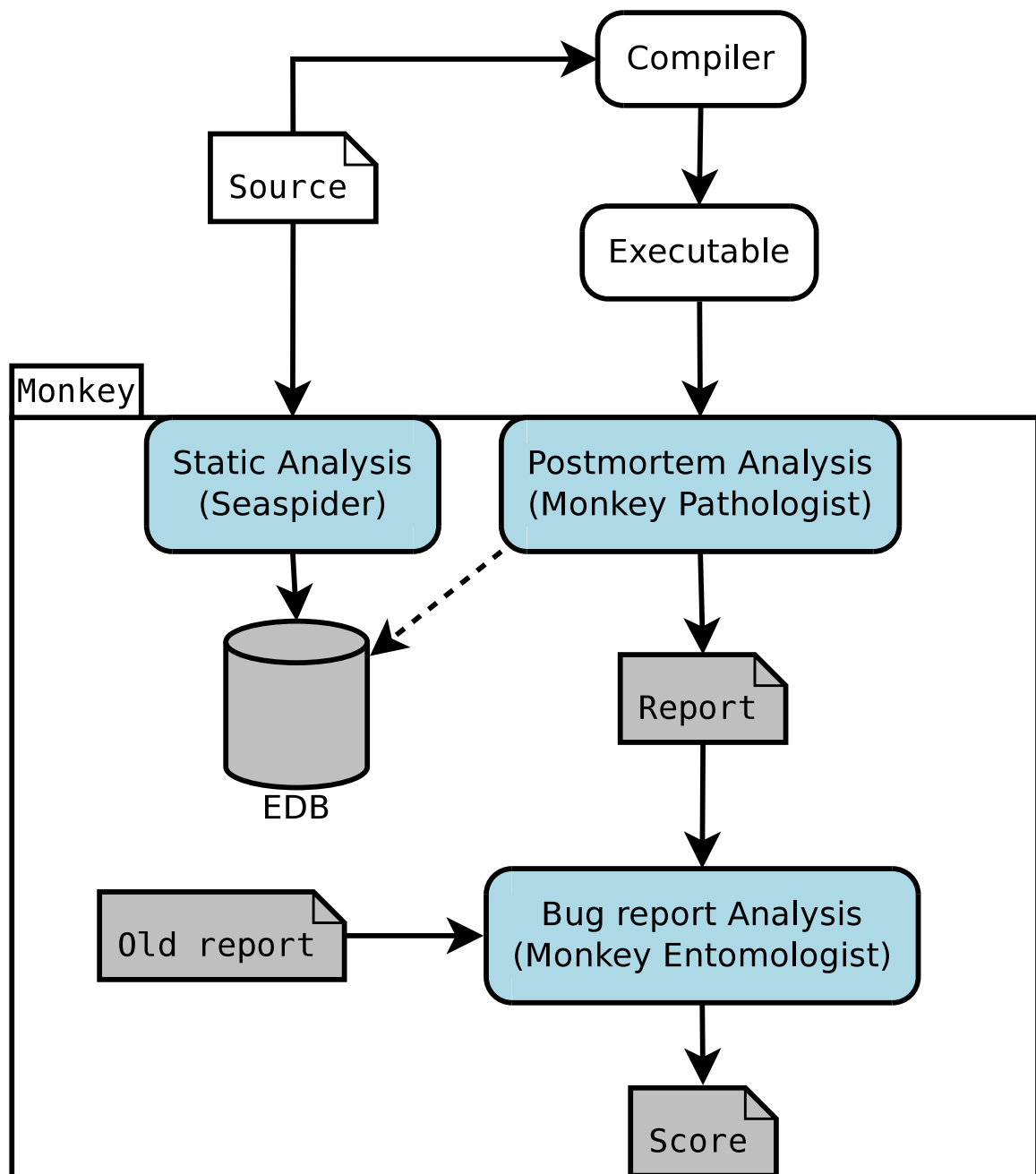


Figure 4.1.: Monkey's architecture.

analyzing the failure reasons (postmortem). It queries the expression database to get the subexpressions of the one where the failure occurred, as well as those expressions residing in the same scope.

The pathologist does its analysis using existent debugging tools. As mentioned earlier, it uses both `gdb`, and `valgrind`. Figure 4.2 highlights the debugging process using Mon-

key pathologist. The scenario shown in the figure is of ARM (gnunet's automatic restart manager). Arm discovers the failure of one of gnunet's services, and uses Monkey to automatically debug that service.

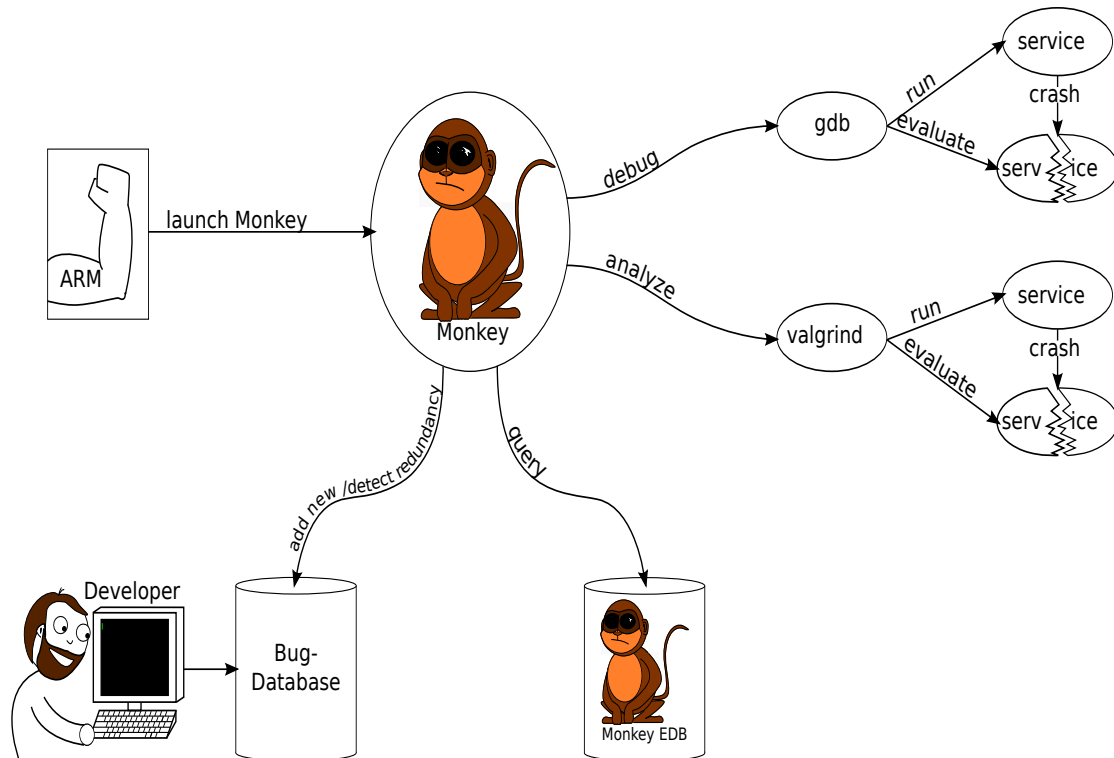


Figure 4.2.: An example of automated debugging with Monkey. Monkey is used for debugging a service whose failure was discovered by gnunet's Automatic Restart Manager (ARM).

The Monkey pathologist produces a bug report as the output. The Monkey bug report is an XML report that contains information about the failure location (file name, function name, line number), and the values of relevant expressions to the one where the failure occurred. The report shows also the program's call trace with values for relevant expressions in each function of the call trace. The pathologist can be configured to run with backtracking enabled. Backtracking means that during its analysis, and after the program fails, the pathologist will run the program in the reverse order. The aim is to examine the values of relevant expressions at many points in time during the execution history, and to show those values in the bug report. Values of relevant expressions at different execution steps may give the developer an insight about the origin of the bug.

By default, Monkey pathologist examines expressions in the same scope of the failed expression. However, it can be configured so that it examines expressions outside that scope.

The XML format in which the Monkey bug report is generated makes it readable and

understandable by both humans and machines. We are particularly interested in generating a machine readable report as one of the services provided by Monkey debugging system is bug report analysis.

4.3. Bug report analysis (The Monkey entomologist)

A typical problem software development teams run into is duplicate bug reports. It is often the case that programmers or software users file the same bug. Manually discovering duplicate reports is a tedious and a time consuming job. In Section 2.2.3, we outlined our proposed idea for solving the bug uniqueness problem by relying on bug characteristics rather than text mining bug descriptions.

In general, it's not easy for a machine to determine bug report redundancies. Even if stack traces are provided, the problem is still compelling. The reason is that stack traces may differ, for the same bug, from one report to another depending on the runtime conditions. Monkey's bug report analysis component (the Monkey entomologist) is our proposed solution for that problem. The entomologist compares two Monkey XML bug reports. It relies on bug characteristics like bug type, file name, line number, call traces, expression values, and `valgrind` reports.

The entomologist is a Java program that takes two Monkey XML bug reports, parses their contents to extract the bug characteristics mentioned above for both bugs. It compares those characteristics, and provides a "Score" on how those two reports (bugs) are similar. The higher the score the comparator gives for the two reports, the higher the likelihood the two reports are for the same bug. Details on Monkey bug comparator scoring mechanism are available in Section 5.4.

5. Implementation

In Chapter 4, we outlined the system architecture, specifying its basic components, the inputs and the outputs of each, and the relationships between them. In this chapter, we will shed light on the implementation of the system components, as well as the overall workflow of the debugging process using Monkey.

5.1. Seaspider

Before starting the debugging process with Monkey, the target program's source tree must be parsed to extract all of its expressions. Seaspider is responsible for parsing the target program's source tree and generating the Monkey expression database. The expression database will be used by Monkey pathologist later while debugging the program.

Seaspider is a standalone program written in Java. Its inputs are the path to the source file/directory of the target program, and the path to the output to be generated after parsing the program: the Monkey expression database file. If a path to a directory is passed to Seaspider as an argument, Seaspider will recursively parse all C files inside that directory, and those inside all of its subdirectories.

Source code parsing is done with the help of `jtb` [21] and `javacc` [16]. `jtb` is a tool that automatically builds an abstract syntax tree for a language grammar. The tool is made to be used together with `javacc`. `javacc` is a Java parser generator: *"A parser generator is a tool that reads a grammar specification and converts it to a Java program that can recognize matches to the grammar."* [16]. `javacc` is an LL(k) bottom-up parser generator. It takes an annotated language grammar file so it can generate a Java parser for that language. The C language parser generated by `javacc` for our C grammar has, however, some limitations: the parser discards the C preprocessing directives, and it expects the resulting source to be syntactically valid.

`jtb` takes a C language grammar file. It generates a set of Java classes, and an annotated grammar file that will be used later by `javacc` to generate the C parser. The Java classes generated by `jtb` are of two types: nodes, and visitors, which implement the interfaces `Node` and `Visitor` respectively. Each `Node` class corresponds a C language construct: e.g. "for statement", "if statement", "function declaration", and so on. The generated visitor classes provide traversal mechanisms for handling the language constructs (the nodes). Node and visitor classes are a typical implementation of the visitor design pattern [10]. Seaspider's main class acts as a client in that implementation of the visitor pattern. `javacc` takes the annotated grammar file and generates a C parser Java class. The parser goes through a C source file and returns the code constructs it finds as a tree of objects of type `Node`.

Seaspider is the program that encapsulates the nodes, the visitors, and the parser classes. Figure 5.1 is a UML class diagram representing Seaspider's static structure. Seaspider has

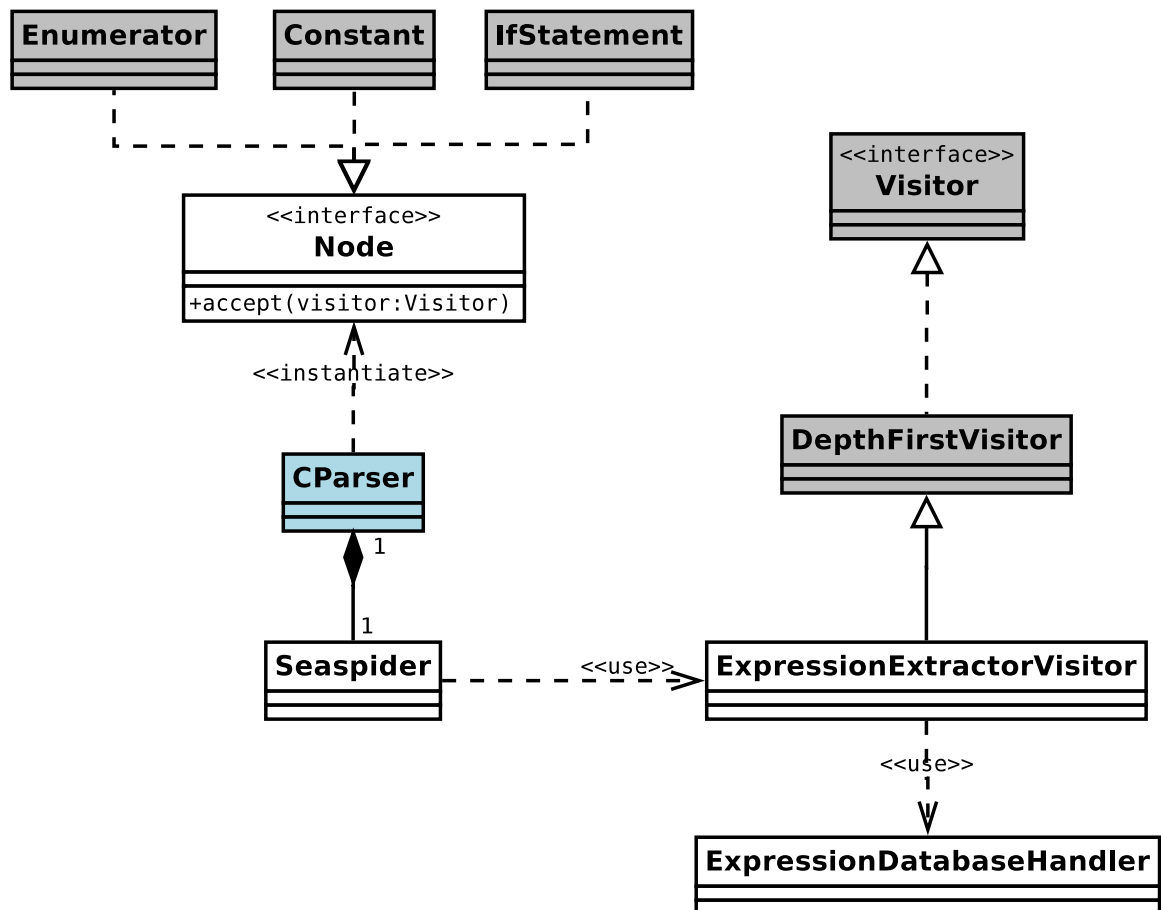


Figure 5.1.: Seaspider UML class diagram. Class in blue is generated by `javacc`, classes and interfaces in gray are generated by `jtb`.

its own visitor class called “ExpressionExtractorVisitor” which extends one of `jtb`’s generated visitor class: “DepthFirstVisitor”. In “ExpressionExtractorVisitor”, visit methods are overridden so that the required functionality is realized. The goal of Seaspider, as mentioned earlier, is to extract the expressions from the target source code. In each visit method, expressions are extracted, and then committed to the expression database. The class “ExpressionDatabaseHandler” is used for creating the Monkey expression database, and for committing expressions to it. Since the Monkey expression database is an `sqlite` database, “ExpressionDatabaseHandler” uses creation and insertion methods provided by `sqljet` [23]. `sqljet` is a Java implementation of the `sqlite` database management system. In the next section we will discuss the Monkey expression database in detail.

Seaspider is shipped as an executable jar generated by compiling the node and visitor classes, the parser class, the `sqljet` jar, and Seaspider’s classes (Figure 5.2 shows Seaspider’s build process).

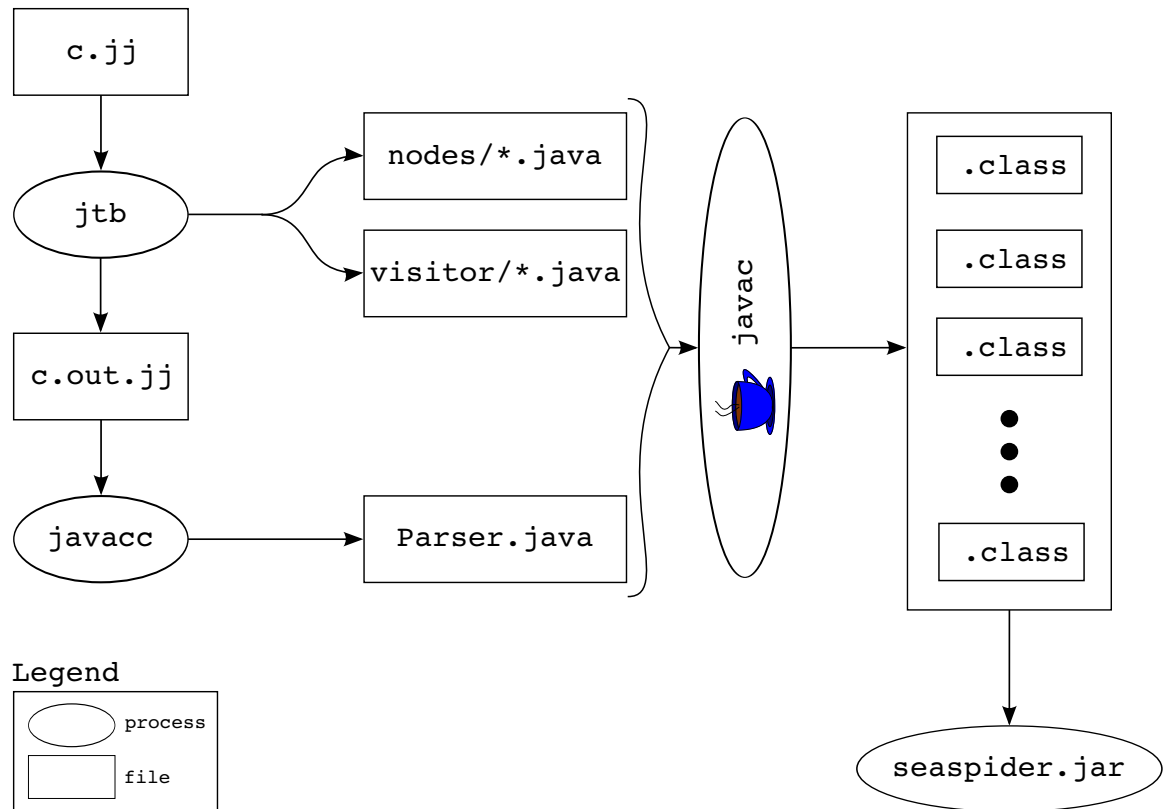


Figure 5.2.: Seaspider’s build process.

5.2. Monkey Expression Database (EDB):

The Monkey expression database is an `sqlite` [24] database with a single table: “Expression”. As mentioned in the previous section, the database is the output from Seaspider.

The table “Expression” contains records for all the expressions in the target program’s source tree. `sqlite` is a lightweight database engine. It provides interfaces for creating and accessing databases in C/C++ and in TCL. For querying the Monkey expression database, Monkey uses the `sqlite` C interface.

For each expression, the following information are extracted by Seaspider and stored into the database:

- **expr_ID**: Expression ID (Primary key).
- **file_name**: File name in which the expression resides.
- **expr_syntax**: Expression Syntax.
- **start_lineno**: Start line number of the expression.
- **end_lineno**: Line number of the expression’s scope end.
- **is_call**: A flag indicating whether the expression contains a function call (The importance of this flag will be explained when discussing Monkey pathologist’s design).

Considering parsing the program “`bug_null_pointer.exception`” shown in Listing 5.1 with Seaspider. The resulting Monkey expression database for this program is shown in Figure 5.3.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 struct CrashStruct {
6     const char *crashValue;
7 };
8
9 void crashFunction()
10 {
11     struct CrashStruct *crashStruct;
12     crashStruct = NULL;
13     printf("Now the program will crash!\n");
14     crashStruct->crashValue = "hello!";
15 }
16
17 int main(int argc, char *argv[])
18 {
19     crashFunction();
20     return 0;
21 }
```

Listing 5.1: `bug_null_pointer.exception.c`: a C program dereferencing a NULL pointer. This program should crash at line 14

In the next section, we will discuss how the Monkey pathologist uses the expression database, and how debugging with Monkey works.

expr_ID	file_name	expr_syntax	start_lineno	end_lineno	is_call
1	bug_null_pointer_exception.c	crashStruct	12	15	0
2	bug_null_pointer_exception.c	NULL	12	15	0
3	bug_null_pointer_exception.c	crashStruct=NULL	12	15	0
4	bug_null_pointer_exception.c	printf("Now the program will crash!\n")	13	15	1
5	bug_null_pointer_exception.c	crashStruct->crashValue	14	15	0
6	bug_null_pointer_exception.c	crashStruct->crashValue="hello!"	14	15	0
7	bug_null_pointer_exception.c	argc	17	21	0
8	bug_null_pointer_exception.c	argv	17	21	0
9	bug_null_pointer_exception.c	crashFunction()	19	21	1

Figure 5.3.: Expression Database contents for the NULL Pointer Exception code snippet.

5.3. Monkey Pathologist

Monkey pathologist is our proposed solution for automated debugging, which is particularly suitable for (but not restricted to) distributed systems. The pathologist is written in C, and its basic inputs are the path to the target program's binary, and the path to the Monkey expression database file for that program. As mentioned above, the target program's Monkey expression database file should have been generated by Seaspider by parsing its source files. The pathologist debugs failed programs by using other debugging tools, currently `gdb` and `valgrind`. The pathologist tries to identify the failure reason by evaluating the relevant expressions to the failure. It communicates with `gdb` or `valgrind` to retrieve relevant information for its analysis. The pathologist generates an XML bug report that provides a detailed analysis including the failure reasons, and the values of relevant expressions. Autonomy is realized as the bug analysis is done without any user intervention.

5.3.1. Strategy

The flow chart in Figure 5.4 shows the pathologist's debugging strategy. It employs post-mortem analysis: it runs the program normally, and it starts its analysis only after the program crashes. Debugging is started with `gdb`. When the program stops (with a failure), the pathologist collects from `gdb` the stack trace information, namely, the received signal, the file name, the function name, and the line number.

At this point, the program's static analysis information collected by Seaspider and available in the program's Monkey expression database become useful. The pathologist queries the expression database to retrieve the expressions relevant to the failure. One subset of relevant expressions are all expressions with the start line numbers equal to the failed expression's line number. Those expressions are precisely the failed expression and its subexpressions. Again, consider the program in Listing 5.1, and its corresponding Monkey expression database shown in Figure 5.3. If the pathologist runs that particular pro-

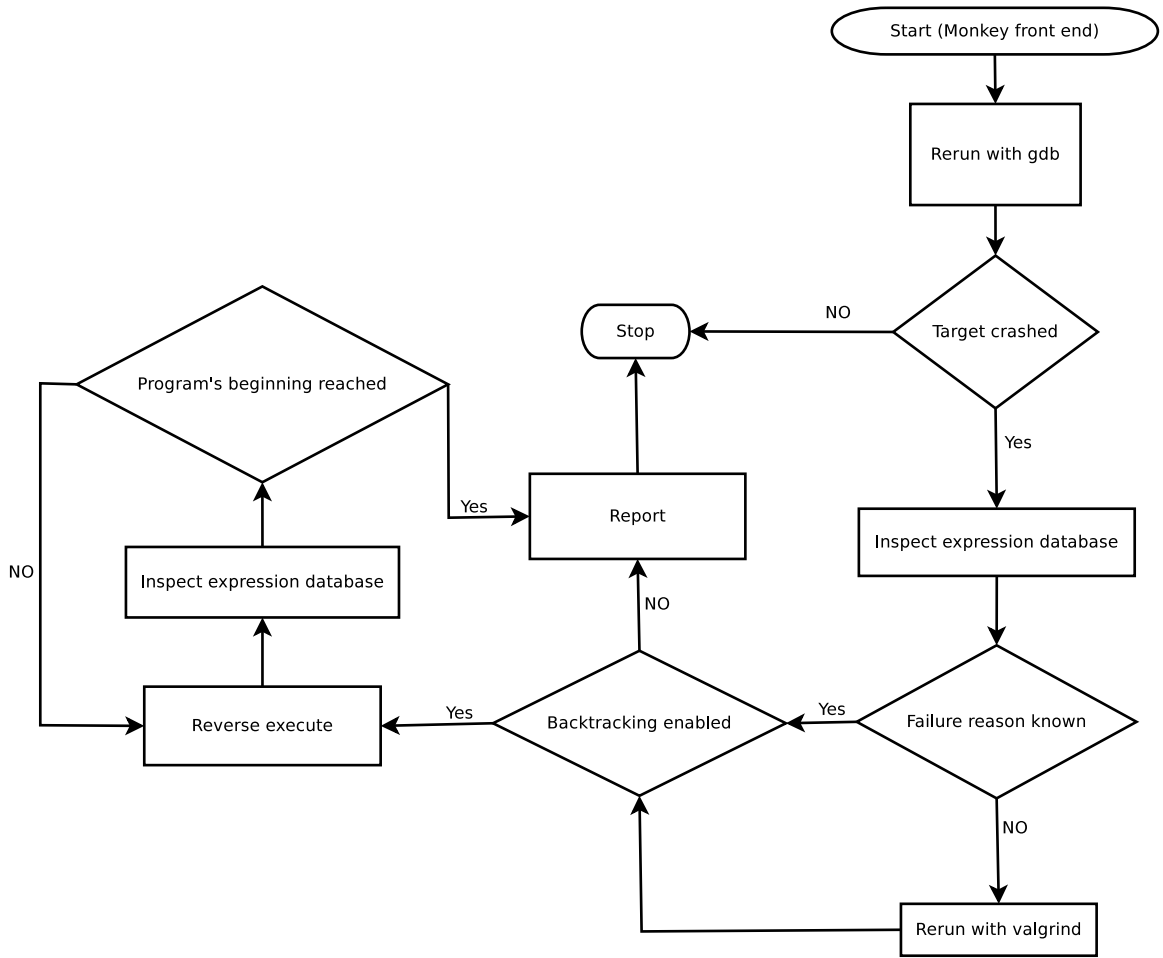


Figure 5.4.: A flow chart describing the pathologist's debugging strategy.

gram, it will crash at line 14 with a SIGSEGV indicating a segmentation fault. Querying the expression database for expressions whose start line number is 14 will result in following two expressions: `crashStruct->crashValue="hello"` (the failed expressions), and `crashStruct->pointer crashValue` (its subexpression).

By default, the pathologist widens the concept of relevant expressions to include all the expressions residing the same scope where the failure occurs. By querying for the failed expression and its subexpressions, it gets their their scope end line number. The pathologist uses this information to retrieve the expressions residing in the failure scope, which are those with the same end line number and whose start line number is less than that of the failed expression (arguably, we are not likely to be interested in what happens after the failure). Back to our example in Listing 5.1 and Figure 5.3, the set of relevant expressions for that bug will be augmented by the expressions: `crashStruct`, `NULL`, `crashStruct=NULL`, the `printf` statement.

By reaching this point in the analysis, the pathologist will be already having the basic bug information and a set of expressions relevant to the failure. The next step will be to get

the values for those expressions, and to try to make sense of those values for a more precise bug classification. In order to do that, the pathologist polls `gdb` for the value of each expression and stores it for later reporting. Some signals, like `SIGSEGV`, may be interpreted in different ways. In the example above, `SIGSEGV` may be the result of a pointer wrongly assigned to `NULL`, or a bad memory access like the use of a free'd memory location. While it might be easy for the user, in our toy problem, to identify the failure reason, the task of bug identification may be more complex in a larger program. In the case of a `SIGSEGV`, the pathologist examines the relevant expression values, and checks whether one of them is set to `NULL`, precisely those which are subexpressions of the failed expression. If that is true, it reports the problem as an NPE (Null pointer exception), providing in its report the value of the `NULL` variable.

Now, considering the program in Listing 5.2. Like the previous example, this program will fail with a `SIGSEGV`, but the reason is different. This time, the `SIGSEGV` is raised because of a bad memory access: the assignment to the unallocated pointer `p` in line 9. The analysis outlined above will get the basic bug information, but the value analysis of the relevant expressions will fail to capture the reason: none of the examined expressions is set to `NULL`. In this particular case, the pathologist will change the debugging tool at runtime. It will trigger `valgrind` for memory analysis. The pathologist launches `valgrind memcheck`, passing to it the path to the program's binary. It will then wait for `valgrind` response, then it will write `valgrind`'s output to the bug report. When receiving a `SIGSEGV` with no `NULL` expressions, the pathologist identifies the bug as a bad memory access, and writes both expression values and a `valgrind`'s output in the bug report. Currently, it changes the debugging tool from `gdb` to `valgrind` when a `SIGSEGV` is received. `SIGILL` is another example of unclear signals for which `valgrind` can be used for further analysis. It's not currently supported though, and will be part of our future work.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 void badMemoryAccess()
6 {
7     int *p = (int*) 0xBAADF00D;
8     printf("Bad memory access now!\n");
9     *p = 5;
10 }
11
12 int main(int argc, char *argv[])
13 {
14     badMemoryAccess();
15     return 0;
16 }
```

Listing 5.2: bug_bad_memory_access.c: a C program attempting to access an unallocated memory location in line 9

Monkey's bug report is in XML format. XML is chosen because it can be easily read by the programmer and it can be easily parsed for further automated analysis: details in Section 5.4. Considering the program in Listing 5.3. The program will fail in line 9 due to an assertion failure. Figure 5.5 shows the program's corresponding Monkey XML bug report. The first tag in the report (the "crash" tag) shows the bug basic information: the bug category, function name, line number, and file name. The report shows then the program's execution history. The execution history is composed of a set of epoch steps. Epoch step 0 means the point in execution history where the program failed. An increase in the epoch step indicates a step back in the execution history. Inside the epoch step, the call trace is shown. Function "assertionFailure" is called by function "main". File name and line numbers of both functions are indicated. The "depth" indicates the order in the call stack: function at depth 0 is called by function at depth 1 and so forth.

```
1 #include <stdio.h>
2 #include <assert.h>
3
4 void assertionFailure()
5 {
6     int x;
7     x = 5;
8     printf("Assertion Failure Now!\n");
9     assert(x < 4);
10 }
11
12 int main(int argc, char *argv[])
13 {
14     assertionFailure();
15     return 0;
16 }
```

Listing 5.3: bug_assertion_failure.c: a C program with an assertion failure in line 9

In Section 5.2, the different fields in the Monkey expression database were explained. The "is_call" field denotes whether the corresponding expression is a function call. The pathologist checks this flag before retrieving the value of a certain expression from `gdb`. If the current expression is a function call, the pathologist does not query its value, and continues with the next expression. The reason is that when querying `gdb` for the value of a function call, `gdb` actually calls the function. During the value analysis phase, the pathologist is not running the program: it only analyzes the values of the relevant expressions. Calling a function in the target program during value analysis may alter the values of other relevant expressions. That might cause a different behavior, and eventually a misleading bug report.

5.3.2. Scope Depth

Scope depth determines the number of scopes in which the pathologist will examine expressions. There are a set of expressions always examined by the pathologist: the set of the sub-expressions of the failed expression. But as mentioned earlier, the pathologist also

```

- <crash category="Assertion Failure" function="assertionFailure" line="9" file="bug_assertion_failure.c">
- <history>
- <epoch step="0">
- <trace>
- <function name="assertionFailure" line="9" file="bug_assertion_failure.c" depth="0">
- <expressions>
- <expression name="assert(x<4)"> Not Evaluated</expression>
- <expression name="x<4"> 0</expression>
- <expression name="printf("Assertion Failure Now!\n")"> Not Evaluated</expression>
- <expression name="x"> 5</expression>
- </expressions>
- </function>
- <function name="main" line="14" file="bug_assertion_failure.c" depth="1">
- <expressions>
- <expression name="assertionFailure()"> Not Evaluated</expression>
- <expression name="argv"> 0xbffff8c4</expression>
- <expression name="argc"> 1</expression>
- </expressions>
- </function>
- </trace>
- </epoch>
- </history>
</crash>

```

Figure 5.5.: Monkey XML bug report for a program with an assertion failure.

examines other expressions. By default, those are the ones residing in the same scope as the failed expression. If that's the case, expression analysis is limited to the failure scope, and we say that the scope depth is 0. In some cases, the analysis provided by scope depth 0 doesn't reveal much information. Increasing scope depth means broadening the scope of the expression analysis. Scope depth determines how far should the pathologist go examining expressions outside the failure scope: scope depth 1 means it should examine expressions in the failure scope and in its immediate outer scope, and scope depth 2 means expression analysis will include 2 scopes outer than the failure scope and so on.

To illustrate how scope depth is important for the analysis accuracy, consider the program shown in Listing 5.4. This program has a division by zero bug in line 14. A SIGFPE will be received in the second loop iteration as the value of the variable *k* will be zero. Now, consider the corresponding Monkey bug report shown in Figure 5.6. The report is the result of running Monkey pathologist with default configuration: scope depth 0. We can see that the pathologist was able to identify the bug type, together with the bug's basic information (function name, line number, file name). But the analysis missed the value of the most relevant expression to the bug: variable *k*. In this example, scope depth 0 is the for loop scope. But variable *k* is defined outside that scope. That's why the pathologist did not examine the value of *k* when running in default mode.

Figure 5.7 shows the Monkey bug report resulting from debugging the program shown in Listing 5.4. This time the pathologist is configured with scope depth 1. The difference is clear: the expression analysis includes the expressions in the superscope of the loop. Most importantly, the value of the variable *k*, which triggered the SIGFPE, is shown in the report.

Note that the "depth=0" XML attribute in the Monkey bug reports shown so far has nothing to do with the scope depth feature. It denotes, as mentioned earlier, the order of

the function in the call trace.

```
1 #include <stdio.h>
2
3 int
4 main (int argc, char *argv[])
5 {
6     int i, k, result, tmp;
7     k = -1;
8     result = 10;
9
10    printf("I am alive!\n");
11    for (i = 0; i < 5; i++)
12    {
13        k += i;
14        result = result / k; //Division by zero in second iteration
15        printf("result = %d\n", result);
16    }
17    return 0;
18 }
```

Listing 5.4: bug_division_by_zero_loop.c: In the second iteration of the loop the program will fail for a division by zero in line 14

```
-<crash category="Division By Zero" function="main" line="14" file="bug_division_by_zero_loop.c">
-  <history>
-    <epoch step="0">
-      <trace>
-        <function name="main" line="14" file="bug_division_by_zero_loop.c" depth="0">
-          <expressions>
-            <expression name="result/k"> Not Evaluated</expression>
-            <expression name="i<5"> 1</expression>
-            <expression name="i" > 1</expression>
-          </expressions>
-        </function>
-      </trace>
-    </epoch>
-  </history>
</crash>
```

Figure 5.6.: Monkey XML bug report for a program with a division by zero bug when Monkey scope depth is zero.

5.3.3. Backtracking

The idea of execution backtracking was outlined in Chapter 2. Backtracking or reverse execution is one of the pathologist's key features. Examining the program's expressions while running it in the reverse order is one of the powerful tools provided by modern

```

-<crash category="Division By Zero" function="main" line="14" file="bug_division_by_zero_loop.c">
-<history>
  -<epoch step="0">
    -<trace>
      -<function name="main" line="14" file="bug_division_by_zero_loop.c" depth="0">
        -<expressions>
          <expression name="result/k"> Not Evaluated</expression>
          <expression name="i<5"> 1</expression>
          <expression name="i"> 1</expression>
          <expression name="printf("I am alive!\n")"> Not Evaluated</expression>
          <expression name="result"> -10</expression>
          <expression name="-1"> -1</expression>
          <expression name="k"> 0</expression>
          <expression name="argv"> 0x7fffffffe688</expression>
          <expression name="argc"> 1</expression>
        </expressions>
      </function>
    </trace>
  </epoch>
</history>
</crash>

```

Figure 5.7.: Monkey XML bug report for a program with a division by zero bug when Monkey scope depth is one.

debugging technologies. It gives a developer, performing manual debugging, a better understanding of the program’s behavior and the failure’s origin. The pathologist makes use of such a tool in `gdb`.

Execution backtracking is not enabled by default: the user has to explicitly enable it (Appendix A discusses Monkey command line arguments in detail). When backtracking is enabled, the pathologist starts the program with `gdb` as usual. The difference in that case is that it issues the `gdb` recording functionality. Currently, recording starts at the target program’s main function: which means the pathologist (effectively `gdb`) records the whole program’s execution. When the program stops, the pathologist starts its postmortem analysis and its expression analysis as usual. The information collected to this point refers to the point in history where the program crashed. We call this point “epoch step 0”. In the default mode of operation, after detecting failure reason, and collecting relevant expression values, the pathologist stops, and generates its report. When reverse execution is enabled, however, it starts (using `gdb`) running the program in the reverse order one line at a time: the pathologist runs the `gdb` command “reverse-next”, which steps back, and through function calls). Running “reverse-next” leads program control to the previous step in execution history, or “epoch step 1”. At this point, the pathologist runs its analysis again: it fetches the expressions relevant to the current expression from the expression database, and examines their values. It keeps executing the target program in the reverse order, and examining relevant expressions until it reaches the beginning of the target program. At this point, the pathologist is ready to generate its bug report that contains backtracking analysis.

Figure 5.8 shows the Monkey XML bug report for the division by zero program in Listing 5.4. The pathologist was run with execution backtracking enabled and with scope

depth equals 1. What is shown in Figure 5.8 is an excerpt of the real report consisting of 9 epoch steps. Unlike the previous report of Figure 5.7, this one consists of multiple epoch steps (just 2 epoch steps are shown), each represents a step in the program's execution history. Epoch 0 shows the values of the relevant expressions when program control is at line 14 (the line where the program failed). Epoch step 1 shows the values of the relevant expressions after one step back (program control is at line 13). The complete report is shown in Appendix A. Backtracking might be especially useful for bugs occurring in loops, as in such cases, the report shows how relevant expression values change with loop iterations.

```
- <crash category="Division By Zero" function="main" line="14" file="bug_division_by_zero_loop.c">
- <history>
- <epoch step="0">
- <trace>
- <function name="main" line="14" file="bug_division_by_zero_loop.c" depth="0">
- <expressions>
  <expression name="result/k"> Not Evaluated</expression>
  <expression name="i<5"> 1</expression>
  <expression name="i"> 1</expression>
  <expression name="printf("I am alive!\n")"> Not Evaluated</expression>
  <expression name="result"> -10</expression>
  <expression name="-1"> -1</expression>
  <expression name="k"> 0</expression>
  <expression name="argv"> 0x7fffffffe688</expression>
  <expression name="argc"> 1</expression>
</expressions>
</function>
</trace>
</epoch>
- <epoch step="1">
- <trace>
- <function name="main" line="13" file="bug_division_by_zero_loop.c" depth="0">
- <expressions>
  <expression name="i<5"> 1</expression>
  <expression name="i"> 1</expression>
  <expression name="printf("I am alive!\n")"> Not Evaluated</expression>
  <expression name="result"> -10</expression>
  <expression name="-1"> -1</expression>
  <expression name="k"> -1</expression>
  <expression name="argv"> 0x7fffffffe688</expression>
  <expression name="argc"> 1</expression>
</expressions>
</function>
</trace>
```

Figure 5.8.: Monkey XML bug report for a program with a division by zero bug. Monkey is run with execution backtracking enabled.

5.3.4. Monkey Pathologist's Components

The pathologist is composed of 4 main components: The pathologist front-end, the action API, the EDB API, and the XML writer. Figure 5.9 shows the pathologist's components and their interactions.

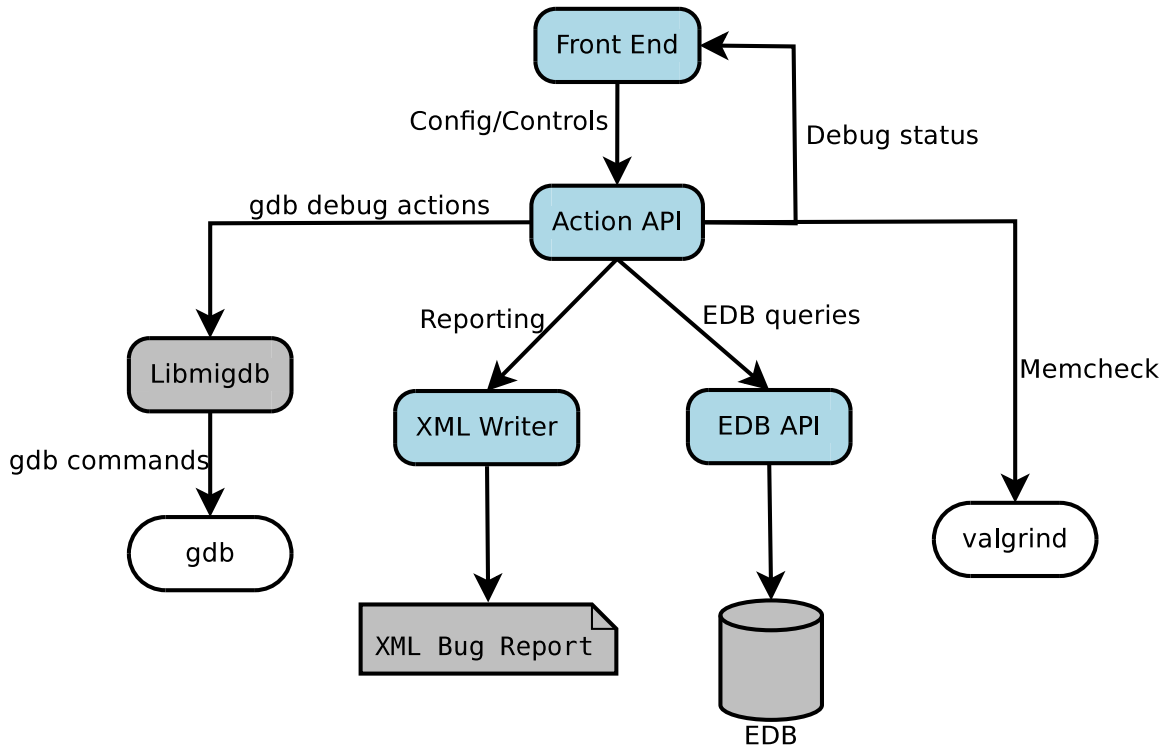


Figure 5.9.: Pathologist's components and their interactions.

- **Pathologist front-end**

The front-end is the pathologist's entry point. It's responsible for command line parsing and for configuration. The front-end is the pathologist's controller: it triggers the debugging actions, then the reporting of the results. It implements the debugging strategy discussed above: first, it starts debugging with `gdb`, and if the analysis results are not conclusive, it launches memory analysis with `valgrind`. According to the given configuration and the debugging status, it calls the reporting function to finally generate the Monkey bug report. The pathologist's different configuration parameters will be discussed in Appendix A. The front-end implements the strategy by calling relevant functions provided by the action API.

- **Action API**

The action API is the component responsible for taking debugging and reporting actions: it launches `gdb` and `valgrind` and runs debug commands. It performs expression analysis, and finally generates bug reports. The action API provides a set

of functions that are called by front-end in order to implement to debugging strategy. The operations exposed by the action API are the following:

- **Rerun with gdb:** This function runs `gdb` and passes to it the path to the target programs binary. If execution backtracking is enabled by user, it starts recording the program's execution. `gdb` starts running the program, and the function synchronously waits for the results. When `gdb` stops, the function analyzes its output, and returns the debug status to the front-end: either the program has stopped normally, or it stopped due to a received signal. If the target program failed, the front-end will call a function for starting expression analysis.
- **Inspect expression database:** The action API queries `gdb` for the values of the relevant expressions. The relevant expressions are collected from the expressions database. The action API uses the EDB API to retrieve the relevant expressions, it then starts analyzing their values. After value analysis, the function identifies the bug type, updates the debugging status and returns it to the front-end.
- **Rerun with valgrind:** According to the debugging status returned after expression analysis, the front-end decides whether a memory analysis is needed. If so, it calls the action API for running `valgrind`. The function launches `valgrind memcheck` giving to it the path to the target program's binary, it reads `valgrind`'s output and saves it to internal data structure to be written later in the bug report.
- **Start reverse execution:** If backtracking is enabled, the front-end will call this function in the action API. The function runs the program in the reverse order. For each step, it performs expressions analysis for the relevant expressions and saves the output for later reporting.
- **Format and write report:** The action API writes the collected debugging information to the Monkey bug report. The report can be generated as text or in XML format. XML reports are used by Monkey entomologist for comparing bugs. The action API uses the XML writer component for formatting and writing the XML report.

- **EDB API**

The expression database API is used to run `sqlite` queries to get relevant expressions. Those expressions are used by the action API for expression value analysis. The EDB API uses `sqlite`'s C library to execute expression retrieval queries on the Monkey's expression database.

- **XML Writer**

5.4. Monkey Entomologist

The entomologist is a Java program that takes two Monkey XML bug reports, compares them, and returns a score representing how similar the two bugs are: the higher the score gets, the higher likelihood the two bugs are the same. The entomologist uses the bug

characteristics written in the XML reports to calculate the similarity score. Figure 5.10 is a class diagram that shows the building blocks of the entomologist.

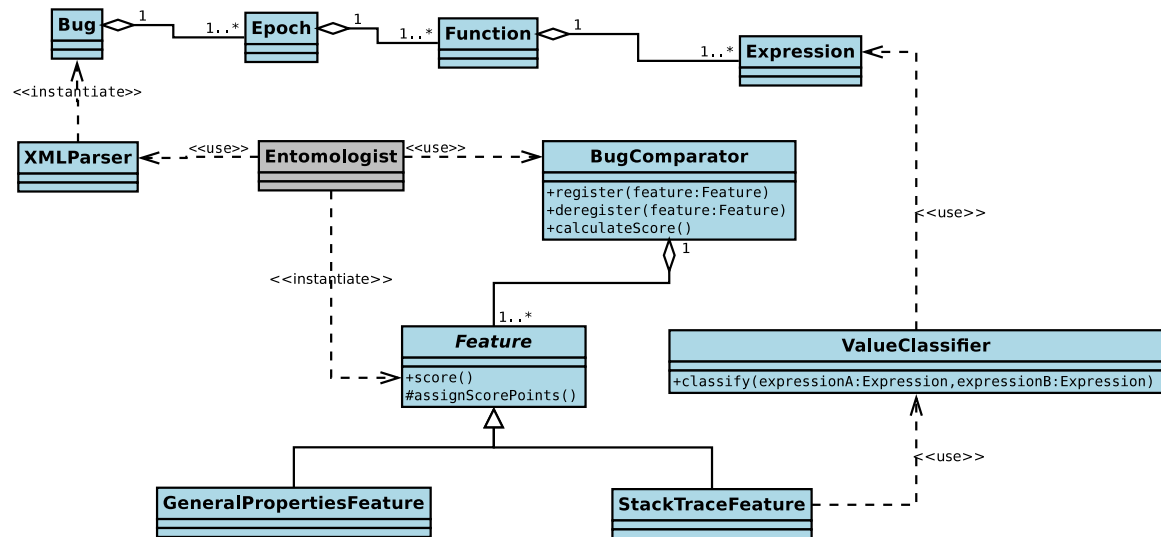


Figure 5.10.: Monkey entomologist class diagram.

The entry point is class **Entomologist**. It uses the **XMLParser** to parse the two XML bug reports. The **XMLParser** then returns an object of type **Bug** for each. A **Bug** object contains the bug characteristics written in the report: each **Bug** is composed of a set of **Epoch** steps (or just one if backtracking was not enabled during debugging), each **Epoch** step is composed of a set of objects of type **Function** (the set of functions represent the call trace in the epoch step), and finally each **Function** is composed of a set of objects of type **Expression**.

Scores are calculated on features, and a final score is the sum of the scores calculated for each feature. The entomologist currently has two features: the general properties feature, and the stack trace feature, represented by classes **GeneralPropertiesFeature** and **StackTraceFeature** respectively. Both classes extend the abstract class **Feature**. Therefore, each **Feature** class has to implement methods `score` and `assignScorePoints`.

Class **Entomologist** uses class **BugComparator** to calculate scores for each of the available features. Class **BugComparator** together with **Feature** classes realize of the observer design pattern [10]: class **Entomologist** registers (or deregisters) features to the **BugComparator**, and then calls **BugComparator**'s `calculateScore` method, which iterates over the registered features and calculate the score of each, adds up the scores, and returns the final score to the **Entomologist**.

Each feature has a configurable top score: comparing two identical bugs will result in top scores for each feature, therefore their similarity score will be of 100%. For example: if the general properties feature's top score is of 100 points and the stack trace feature's top score is 50 points, the two identical bugs' overall score will be of 150 of 150 which is 100%. The entomologist configuration file is a Java properties file that can be edited by the user to set the features's scores: Appendix A shows the properties files' contents.

Each feature has its own properties. A feature's score is the weighted sum of its properties' scores. The general properties feature, for example, has the following properties: Bug category, line number, function name, file name. The user can give weights to those properties according to their importance: how much he wants a property to contribute to the feature's score. For example: a user can give them equal weights, so each property will have a weight of 0.25. Again with the two identical bugs, the comparator will check their bug category, line number, function name, and file name, find the two bugs have same values for all properties, therefore, the comparison will result a top score for the general properties feature.

For the stack trace feature, there are two main properties: function characteristics, and the expressions. The stack trace is composed of a set functions, and each function is composed of a set of expressions. Weighting the function's characteristics means how much to value the function's name, the function's line number, and the function's file name regardless of their contents. Expressions' weight is the weight of expressions syntax and values. The user may select giving a higher weight for expressions and a lower weight for function characteristics. That means what highly matters is the contents of the stack trace, and not the names of the functions and their locations in the source code. Such a configuration will make entomologist giving high scores for similar bugs despite code changes: if the code was changed without fixing the bug, and a report was filed for the same bug after that code change, then the entomologist, with that configuration, will still be able to give high score for the two reports despite the changes that might occur in the second report in the program's expressions and stack trace.

The stack trace feature properties have their own configurable sub-properties. For the function characteristics property, the user can give independent weights to function name, line number, and file name: a reasonable selection could be to give higher weight for file name, but lower weights for function name, and line number. The expression property has two sub-properties: expression syntax, and expression values.

The contribution of the expression values to the similarity score has its drawbacks. Expression values may differ between bug reports for reasons that are not related to the bug, for example, the runtime environment or the programs inputs. Two bug reports can be identical despite having completely different expression values. To solve this problem, the entomologist does not directly compare expression values. Instead, it compares expression value classes. For each of the types: integers, pointers, floating point numbers, and strings, the entomologist defines value classes. They are as follows:

- Integers
 - 0
 - 1
 - MAXINT
 - MININT
 - -1
 - All other integer values
- Pointers

- NULL
 - All other pointer values
- Floats
 - 0.0
 - All other floating point values
- Strings
 - NULL
 - empty string
 - All other string values

For integers, values belonging to the same value classes are equivalent, and therefore treated by the entomologist as equals and scored accordingly. For example, if in bug report A variable *x* has the value 5, while in bug report B *x* has the value 10, the entomologist will score it as if *x* has the same value in both reports, because 5 and 10 belong to the same value class. But, if *x* is 0 in one report and 10 in the other, the entomologist will decide that *x* is not equal in the two reports, and score them accordingly. In integers, the values 0, 1, MAXINT, MININT, and -1 are special values: they are usually used to indicate program status, or error messages, MAXINT, and MININT are round numbers whose occurrence might be alarming and related to the problem. That's why the entomologist does compare those values. The same concept applies for the other types.

When comparing two corresponding expressions from two bug reports, class `StackTraceFeature` calls class `ValueClassifier`'s method `classify`, which takes the two expressions, compares their values according to the above discussion, and returns a boolean value telling `StackTraceFeature` whether the two expressions are equal.

Part III.

Results and Conclusion

6. Runtime Examples

In this chapter, a step-by-step demonstration will show how to run the different Monkey components (Seaspider, Monkey pathologist, and Monkey entomologist) on examples of buggy programs. Debugging results, and report comparison results will be shown and discussed.

6.1. Demonstration Overview

In Chapter 5, we've shown and discussed some examples of XML bug reports generated by Monkey pathologist. In this demonstration, we will show the steps of using Monkey pathologist to debug programs and generate XML bug reports. Then, we will show different cases of using the Monkey entomologist to measure similarity between bugs. The demonstration will start by running Monkey pathologist on a buggy program which will generate a bug report, then, the buggy program will be modified in a way that alters the program's stack trace and expressions without changing the failure reason. Then, we will debug the modified program again with pathologist and generate another report. We will run the entomologist and see that despite the change in the source code, the two reports are having a high similarity score. We will change the buggy program again, so that it fails for another reason, and we will generate new bug report. We will compare the reports with entomologist again, and see that in the case of different bugs, the similarity score is low.

6.1.1. Example: NULL pointer dereference

Let's consider the C program shown in Listing 6.1. This program dereferences a NULL pointer in line 14, therefore, it will fail with a segmentation fault. We are going to debug this program with Monkey pathologist.

Parsing source code with Seaspider

Before launching Monkey pathologist, we need to build the Monkey expression database for the program's source code. We will use Seaspider to parse the code and generate the database. To do so, we will run the following command:

```
$ seaspider bug_null_pointer_exception.c  
    bug_null_pointer_exception.db
```

Generating Reports with Monkey pathologist

We need to build the source code to generate the program's binary. We will use `gcc` with options to disable optimization, and enable producing debugging information:

```
$ gcc -g -O0 -o bug_null_pointer_exception
    bug_null_pointer_exception.c
```

Having the binary generated, we will start debugging the program with Monkey pathologist by running the command:

```
$ monkey -m text -d bug_null_pointer_exception.db -o npe_report.
    xml -b
bug_null_pointer_exception
```

“-m” specifies the report type: available options are “text”, and “email”. “-d” specifies the path to the expression database. “-b” for the binary, and “-o” for the output XML report.

Figure 6.1 shows the resulting XML bug report. The report starts with the bug characterization: category (NULL pointer exception), file name, and line number. The reports shows the values of the relevant expressions (“crashStruct” has the value “0x0”).

Comparing bug reports

High score We will modify the code in Listing 6.1 so that the same bug is reproduced by a slightly different stack trace. The modified code is shown in Listing 6.2. Instead of calling “crashFunction()” directly by “main()”, “main()” will call “intermediateFunction()”, and then “intermediateFunction()” will call “crashFunction()” where the failure occurs. “crashFunction()” itself is modified by writing some additional expressions in it. Again, we will parse the modified code with Seaspider to generate a new expression database:

```
$ seaspider bug_null_pointer_exception.c
    bug_null_pointer_exception_modified.db
```

Then, we will build the source code with gcc:

```
$ gcc -g -O0 -o bug_null_pointer_exception
    bug_null_pointer_exception.c
```

And start debugging the modified program with Monkey:

```
$ monkey -m text -d bug_null_pointer_exception_modified.db -o
npe_modified_report.xml -b bug_null_pointer_exception
```

Figure 6.2 shows the new XML report generated by Monkey. The new report is different from the original one: a new function call is added to the call trace, and new expressions are added to function “crashFunction()”.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 struct CrashStruct {
6     const char *crashValue;
7 };
8
9 void crashFunction()
10 {
```

```
11 struct CrashStruct *crashStruct;
12 crashStruct = NULL;
13 printf("Now the program will crash!\n");
14 crashStruct->crashValue = "hello!";
15 }
16
17 int main(int argc, char *argv[])
18 {
19     crashFunction();
20     return 0;
21 }
```

Listing 6.1: `bug_null_pointer_exception.c`: a C program dereferencing a NULL pointer. This program should crash at line 14

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 struct CrashStruct {
6     const char *crashValue;
7 };
8
9 void crashFunction()
10 {
11     struct CrashStruct *crashStruct;
12     int a;
13     int b;
14     int c;
15     a = 3;
16     b = 5;
17     c = a + b;
18     crashStruct = NULL;
19     printf("Now the program will crash!\n");
20     crashStruct->crashValue = "hello!";
21 }
22
23 void intermediateFunction()
24 {
25     crashFunction();
26 }
27
28 int main(int argc, char *argv[])
29 {
30     intermediateFunction();
31     return 0;
```

32 }

Listing 6.2: `bug_null_pointer_exception.c`: a C program dereferencing a NULL pointer. Previous program is modified by adding `"intermediateFunction()"` to the call trace and adding new expressions to `"crashFunction()"`

The next step will be to compare the reports in Figure 6.1, and Figure 6.2 using Monkey entomologist. The output of the entomologist is a score (percentage) representing how similar the two bugs are. The entomologist is packaged as an executable jar. Figure 6.3 shows the command and the similarity score. Monkey bug comparator gives a high score (about 82%) despite the differences between the two programs in the call trace and the examined expressions. The entomologist's feature scores, and properties' weights were assigned in the entomologist's configuration file (a Java properties file). The entomologist's configuration used in the examples of this chapter are shown in Appendix A.

Low score The NULL pointer dereference program in Listing 5.1 is modified again in Listing 6.3. This time another bug is seeded to the program before dereferencing the NULL pointer. While executing the second loop iteration, the program will fail in line 21 because of an attempt to divide by zero: in the second loop iteration, variable `"k"` will have the value zero. As shown before, the new program will be parsed by Seaspider for producing its Monkey expression database. The program will be compiled with `gcc`, and we will start debugging it with Monkey pathologist by running the command:

```
$ monkey -m text -d bug_null_pointer_exception_divzero.db -o  
npe_divzero_report.xml -p 1 -b bug_null_pointer_exception
```

The only difference between this command and the previous ones is the use of the option `"-p 1"`. This option tells Monkey that if the failure occurs in a nested scope, it should not restrict examining subexpressions in that scope. Instead, it should examine expressions in outer scopes too. The value `"1"` means that Monkey should inspect expressions in only one outer scope. Figure 6.4 shows the resulting XML bug report.

The NULL pointer dereference program fails this time for another reason: a division by zero. The reports shown in Figure 6.1, and Figure 6.4 have some similarities: same file name, same function name, same call trace, some similarities in the examined expressions and their values. But, each is reporting a different bug. We should only expect a low score from Monkey entomologist when comparing those two reports. Figure 6.5 shows the result of running the entomologist on the two bug reports. The comparison shows they are only about 32% similar.

```
1 #include <stdio.h>  
2 #include <string.h>  
3  
4  
5 struct CrashStruct {  
6     const char *crashValue;  
7 };  
8  
9 void crashFunction()
```

```
10 {
11     struct CrashStruct *crashStruct;
12     int i, k, result;
13     crashStruct = NULL;
14     k = -1;
15     result = 10;
16
17     printf("I am alive!\n");
18     for (i = 0; i < 5; i++)
19     {
20         k += i;
21         result = result / k;
22         printf("result = %d\n", result);
23     }
24     printf("Now the program will crash!\n");
25     crashStruct->crashValue = "hello!";
26 }
27
28 int main(int argc, char *argv[])
29 {
30     crashFunction();
31     return 0;
32 }
```

Listing 6.3: bug_null_pointer_exception.c: a division by zero bug is seeded to crashFunction

```

- <crash category="npe" function="crashFunction" line="14" file="bug_null_pointer_exception.c">
  <history>
    <epoch step="0">
      <trace>
        <function name="crashFunction" line="14" file="bug_null_pointer_exception.c" depth="0">
          <expressions>
            <expression name="crashStruct->crashValue="hello!"> Not Evaluated</expression>
            <expression name="crashStruct->crashValue"> Not Evaluated</expression>
            <expression name="printf("Now the program will crash!\n")"> Not Evaluated</expression>
            <expression name="NULL"> 0x0</expression>
            <expression name="crashStruct"> 0x0</expression>
          </expressions>
        </function>
      <function name="main" line="19" file="bug_null_pointer_exception.c" depth="1">
        <expressions>
          <expression name="crashFunction()"> Not Evaluated</expression>
          <expression name="argv"> 0xbffff384</expression>
          <expression name="argc"> 1</expression>
        </expressions>
      </function>
    </trace>
  </epoch>
</history>
</crash>

```

Figure 6.1.: Monkey bug report for a program dereferencing a NULL pointer.

```

- <crash category="npe" function="crashFunction" line="20" file="bug_null_pointer_exception.c">
- <history>
- <epoch step="0">
- <trace>
- <function name="crashFunction" line="20" file="bug_null_pointer_exception.c" depth="0">
- <expressions>
  <expression name="crashStruct->crashValue="hello!"> Not Evaluated</expression>
  <expression name="crashStruct->crashValue"> Not Evaluated</expression>
  <expression name="printf("Now the program will crash!\n")"> Not Evaluated</expression>
  <expression name="NULL"> 0x0</expression>
  <expression name="a+b"> 8</expression>
  <expression name="c"> 8</expression>
  <expression name="b"> 5</expression>
  <expression name="a"> 3</expression>
  <expression name="crashStruct"> 0x0</expression>
</expressions>
</function>
- <function name="intermediateFunction" line="25" file="bug_null_pointer_exception.c" depth="1">
- <expressions>
  <expression name="crashFunction()"> Not Evaluated</expression>
</expressions>
</function>
- <function name="main" line="30" file="bug_null_pointer_exception.c" depth="2">
- <expressions>
  <expression name="intermediateFunction()"> Not Evaluated</expression>
  <expression name="argv"> 0xbffff374</expression>
  <expression name="argc"> 1</expression>
</expressions>
</function>
</trace>
</epoch>
</history>
</crash>

```

Figure 6.2.: Monkey bug report for a program dereferencing a NULL pointer. The program is modified by adding some new expressions to alter the stack trace.

```

> ./entomologist npe.xml npe_modified_report.xml -p comparator.properties
Similarity Percentage = 81.87393162393163
>

```

Figure 6.3.: Similarity score for two XML bug reports describing the same bug. Despite the difference in stack trace and expressions between the two reports, the score is still high.

```

-<crash category="Division By Zero" function="crashFunction" line="21" file="bug_null_pointer_exception.c">
  -<history>
    -<epoch step="0">
      -<trace>
        -<function name="crashFunction" line="21" file="bug_null_pointer_exception.c" depth="0">
          -<expressions>
            <expression name="crashStruct->crashValue"> Not Evaluated</expression>
            <expression name="printf("Now the program will crash!\n")"> Not Evaluated</expression>
            <expression name="printf("result = %d\n",result)"> Not Evaluated</expression>
            <expression name="result/k"> Not Evaluated</expression>
            <expression name="i<5"> 1</expression>
            <expression name="printf("I am alive!\n")"> Not Evaluated</expression>
            <expression name="-1"> -1</expression>
            <expression name="NULL"> 0x0</expression>
            <expression name="result"> -10</expression>
            <expression name="k"> 0</expression>
            <expression name="i"> 1</expression>
            <expression name="crashStruct"> 0x0</expression>
          </expressions>
        </function>
      -<function name="main" line="30" file="bug_null_pointer_exception.c" depth="1">
        -<expressions>
          <expression name="crashFunction()"> Not Evaluated</expression>
          <expression name="argv"> 0xbffff374</expression>
          <expression name="argc"> 1</expression>
        </expressions>
      </function>
    </trace>
  </epoch>
</history>
</crash>

```

Figure 6.4.: Monkey bug report for a program that fails with a division by zero.

```

>./entomologist npe.xml npe_divzero_report.xml -p comparator.properties
Similarity Percentage = 32.358333333333334
>

```

Figure 6.5.: Similarity score for two XML bug reports describing two different bugs. Despite some similarities in examined expressions and call trace, the score is low.

7. Discussion and Future Work

Training a monkey is always unfinished business. There are several areas in which Monkey is subject to improvements, which can lead to a better bug characterization and a more accurate report triaging. In this Chapter, we will introduce some ideas for future improvements.

7.1. Presentation

Currently, Monkey's bug report is in XML. The motivation behind generating reports in XML format was to make report parsing easier. However, the report content can be presented to the developer in a more user friendly format. One idea is to create a web based tool that annotates the information provided by the report to the target program's source code. Using the web tool, the developer can view the available Monkey bug reports. When he opens a report, the tool will load the target program's source code, and highlights the relevant expressions to the bug in the source code. The developer can view the values of the relevant expressions by mouse hovering on the highlighted expressions.

Another idea is to integrate Monkey reports to the source code in an integrated development environment. For example, a Monkey `eclipse` [9] plugin can be created. Using the plugin, the developer can either trigger the pathologist to debug a program, or to read the relevant expression values of an already available Monkey report annotated to the source code.

7.2. Monkey Pathologist

Currently, backtracking is only enabled for the whole program, which means the execution is recorded from the beginning until the program crashes. Enabling backtracking only for the critical parts of the target program will lead to a faster execution and a reduced memory consumption. This can be realized by allowing the developer to annotate the source code in the areas where he wants backtracking to be enabled. The annotations can then be parsed by Seaspider, and inserted into the expression database. At runtime, the pathologist can read those annotations and enable or disable the execution recording/backtracking accordingly.

Reporting can be improved by enabling the pathologist to submit bug reports directly to bug tracking systems like `bugzilla` [8] or `mantis` [5].

Bug characterization is currently limited to four types of bugs: null pointer exception, bad memory access, division by zero, and assertion failure. We are planning to develop Seaspider's static analysis, and the pathologist's value analysis to characterize new bug categories. We are particularly interested in the areas of data races and deadlocks.

Debugging is currently restricted to C programs, we plan to support other languages in the future.

7.3. Monkey Seaspider

Seaspider extracts all the expressions from the target program's source code and inserts their syntax and locations into the database. Seaspider can be updated to perform interprocedural analysis, which can give information about function pointers. When the pathologist detects a function pointer, it can place a breakpoint in the function's body, examine the values of the expressions in the function, and add them to the bug report.

Seaspider can also support inter and intra procedural flow analysis. Having such information in the Monkey database, the pathologist can examine only those expressions that are relevant to the execution, instead of examining all the expressions in a scope.

7.4. Monkey Entomologist

The current implementation of the entomologist provides a proof of concept of bug triaging based on bug characteristics. However, statistics are required to find the optimal tuning of the feature weights, and to set a score threshold that decides report redundancy. That can be realized by creating a database of Monkey reports, running the entomologist on pairs of reports, tune the feature weights, and analyze scores.

Report clusters can be created. A cluster will represent a group of reports whose pairwise scores assigned by the entomologist were the highest. An incoming report can be compared to the central report of each of the available clusters for fast categorization.

7.5. Conclusion

A software developer should focus on writing new software and solving challenging problems rather than wasting his time investigating runtime behavior, a task that can and should be automated. Monkey represents the first step towards an intelligent debugging process in which both debugging and filing bug reports are fully automated, the content of a bug report is accurate and guides the developer to fixing the problem with minimal further investigation, and that ensures that a bug report repository is not inflated with duplicate reports whose triaging might stall the software team's productivity.

Appendix

A. Appendix

A.1. Downloading and installing Monkey

Monkey is available under the GNU GPL for GNU/Linux systems. Monkey requires the GNU C compiler, the GNU autotools, the GNU debugger, a Java Development Kit ≥ 5.0 (for the Seaspider and the Monkey entomologist components), `sqlite3` (for accessing the Monkey database), and an optional `libesmtp` to enable sending bug reports as e-mails. On systems where the dependencies are satisfied, you can compile, install and run Monkey as follows:

```
$ svn checkout https://gnunet.org/svn/monkey/
$ cd monkey; ./bootstrap ;
$ ./configure ; make install
$ seaspider -d /tmp/expression.db path/to/your/c/code
$ monkey -e /tmp/expression.db -m bugs@example.com COMMAND ARGS
...
$ entomologist report1.xml report2.xml -p comparator.properties
```

A.2. Monkey pathologist's command arguments

The Monkey pathologist has the following command arguments:

- `-b`: binary: path to a binary file for a program to debug with monkey [mandatory]
- `-d`: database: path to Expression Database file [mandatory]
- `-m`: mode: monkey's mode of operation: options are "text" or "email" [mandatory]
- `-a`: address: email address to which monkey will send debug report [optional]
- `-f`: function: function name at which monkey will set a breakpoint [optional]
- `-g`: gdb: path to gdb binary to use. If not passed, monkey will set it to `/usr/bin/gdb` [optional]
- `-l`: valgrind: path to valgrind binary. If not passed, monkey will set it to `/usr/bin/valgrind` [optional]
- `-p`: depth: scope depth. How many scopes should monkey lookup expression values outer than the scope in which the problem occurs. Default is 0 which means, monkey is restricted to the scope of the expression in which the debugged program stopped [optional]

- -r: reverse: enables reverse execution. By default it's not allowed [optional]
- -o: output: path to the debug report generated by monkey. If not provided monkey will save the report in a file named monkey_output in the current working directory [optional]
- -i: inspect: expression to inspect in the function specified after argument f [this option is not currently supported]
- -v: version: monkey's version [optional]
- -h: help: print this help [optional]

A.3. Entomologist's configuration

As discussed in Chapter 5, the entomologist is configured using a Java properties file. Typical contents of an entomologist's properties file are as follows:

```
MAX_INT=2147483647
MIN_INT=-2147483647
ENABLE_VALUE_CLASSES=false
LINE_NUMBER_DIFFERENCE_THRESHOLD=10

# Features' scores
GENERAL_PROPERTIES_FEATURE_SCORE=100
STACK_TRACE_FEATURE_SCORE=50

# Weights for feature: Bug General Properties
BUG_CATEGORY_WEIGHT=0.75
LINE_NUMBER_WEIGHT=0.00625
BUG_FUNCTION_NAME_WEIGHT=0.1875
FILE_NAME_WEIGHT=0.05625

# Weights for feature: Stack-Trace
# Weight of all function semantics in the stack-trace
FUNCTION_SEMANTICS_WEIGHT=0.1
# Weight of all expressions in the stack-trace
EXPRESSIONS_WEIGHT=0.9

# Within the function semantics
FUNCTION_NAME_WEIGHT=0.25
FUNCTION_LINE_WEIGHT=0.25
FUNCTION_FILE_NAME=0.5

# Within the expressions of the function
EXPRESSION_SYNTAX_WEIGHT=0.7
EXPRESSION_VALUE_WEIGHT=0.3
```

The entomologist's configuration in this example was used to calculate the similarity scores shown in Chapter 6. The properties file defines the score points of the registered features, the weights of each of the features' properties, and the weights of the sub-properties of the stack trace feature.

A.4. Bug reports

A.4.1. Division by zero bug with backtracking enabled

In Chapter 5, we discussed the idea of execution backtracking by giving the following example:

```
1 #include <stdio.h>
2
3 int
4 main (int argc, char *argv[])
5 {
6     int i, k, result, tmp;
7     k = -1;
8     result = 10;
9
10    printf("I am alive!\n");
11    for (i = 0; i < 5; i++)
12    {
13        k += i;
14        result = result / k; //Division by zero in second iteration
15        printf("result = %d\n", result);
16    }
17    return 0;
18 }
```

Listing A.1: bug_division_by_zero_loop.c: In the second iteration of the loop the program will fail for a division by zero in line 14

Running Monkey pathologist with backtracking enabled and with scope depth equals 1 results in the following XML bug report:

```
<?xml version="1.0"?>
<crash category="Division By Zero" function="main" line="14" file=
  ="bug_division_by_zero_loop.c">
  <history>
    <epoch step="0">
      <trace>
        <function name="main" line="14" file="
          bug_division_by_zero_loop.c" depth="0">
            <expressions>
              <expression name="result/k"> Not Evaluated</expression>
              <expression name="i<5">1</expression>
```

```
<expression name="i">1</expression>
<expression name="printf("I am alive!\n")">Not Evaluated
  </expression>
<expression name="result">-10</expression>
<expression name="-1">-1</expression>
<expression name="k">0</expression>
<expression name="argv">0x7ffffffffffe688</expression>
<expression name="argc">1</expression>
</expressions>
</function>
</trace>
</epoch>
<epoch step="1">
  <trace>
    <function name="main" line="13" file="
      bug_division_by_zero_loop.c" depth="0">
      <expressions>
        <expression name="i<5">1</expression>
        <expression name="i">1</expression>
        <expression name="printf("I am alive!\n")">Not Evaluated
          </expression>
        <expression name="result">-10</expression>
        <expression name="-1">-1</expression>
        <expression name="k">-1</expression>
        <expression name="argv">0x7ffffffffffe688</expression>
        <expression name="argc">1</expression>
      </expressions>
    </function>
  </trace>
</epoch>
<epoch step="2">
  <trace>
    <function name="main" line="11" file="
      bug_division_by_zero_loop.c" depth="0">
      <expressions>
        <expression name="i<5">1</expression>
        <expression name="i">0</expression>
        <expression name="printf("I am alive!\n")">Not Evaluated
          </expression>
        <expression name="result">-10</expression>
        <expression name="-1">-1</expression>
        <expression name="k">-1</expression>
        <expression name="argv">0x7ffffffffffe688</expression>
        <expression name="argc">1</expression>
      </expressions>
    </function>
```

```

    </trace>
  </epoch>
  <epoch step="3">
    <trace>
      <function name="main" line="15" file="
        bug_division_by_zero_loop.c" depth="0">
        <expressions>
          <expression name="printf("result = %d\n",result)">Not
            Evaluated</expression>
          <expression name="result/k">10</expression>
          <expression name="i<5">1</expression>
          <expression name="i">0</expression>
          <expression name="printf("I am alive!\n")">Not Evaluated
            </expression>
          <expression name="result">-10</expression>
          <expression name="-1">-1</expression>
          <expression name="k">-1</expression>
          <expression name="argv">0x7fffffff688</expression>
          <expression name="argc">1</expression>
        </expressions>
      </function>
    </trace>
  </epoch>
  <epoch step="4">
    <trace>
      <function name="main" line="14" file="
        bug_division_by_zero_loop.c" depth="0">
        <expressions>
          <expression name="result/k">-10</expression>
          <expression name="i<5">1</expression>
          <expression name="i">0</expression>
          <expression name="printf("I am alive!\n")">Not Evaluated
            </expression>
          <expression name="result">10</expression>
          <expression name="-1">-1</expression>
          <expression name="k">-1</expression>
          <expression name="argv">0x7fffffff688</expression>
          <expression name="argc">1</expression>
        </expressions>
      </function>
    </trace>
  </epoch>
  <epoch step="5">
    <trace>
      <function name="main" line="13" file="
        bug_division_by_zero_loop.c" depth="0">

```

```
<expressions>
  <expression name="i<5">1</expression>
  <expression name="i">0</expression>
  <expression name="printf("I am alive!\n")">Not Evaluated
    </expression>
  <expression name="result">10</expression>
  <expression name="-1">-1</expression>
  <expression name="k">-1</expression>
  <expression name="argv">0x7fffffffe688</expression>
  <expression name="argc">1</expression>
</expressions>
</function>
</trace>
</epoch>
<epoch step="6">
  <trace>
    <function name="main" line="11" file="
      bug_division_by_zero_loop.c" depth="0">
      <expressions>
        <expression name="i<5">1</expression>
        <expression name="i">0</expression>
        <expression name="printf("I am alive!\n")">Not Evaluated
          </expression>
        <expression name="result">10</expression>
        <expression name="-1">-1</expression>
        <expression name="k">-1</expression>
        <expression name="argv">0x7fffffffe688</expression>
        <expression name="argc">1</expression>
      </expressions>
    </function>
  </trace>
</epoch>
<epoch step="7">
  <trace>
    <function name="main" line="10" file="
      bug_division_by_zero_loop.c" depth="0">
      <expressions>
        <expression name="printf("I am alive!\n")"> Not
          Evaluated</expression>
        <expression name="result">10</expression>
        <expression name="-1">-1</expression>
        <expression name="k">-1</expression>
        <expression name="argv">0x7fffffffe688</expression>
        <expression name="argc">1</expression>
      </expressions>
    </function>
```

```
</trace>
</epoch>
<epoch step="8">
  <trace>
    <function name="main" line="8" file="
      bug_division_by_zero_loop.c" depth="0">
      <expressions>
        <expression name="result"> 32767</expression>
        <expression name="-1">-1</expression>
        <expression name="k">-1</expression>
        <expression name="argv">0x7ffffffffffe688</expression>
        <expression name="argc">1</expression>
      </expressions>
    </function>
  </trace>
</epoch>
<epoch step="9">
  <trace>
    <function name="main" line="7" file="
      bug_division_by_zero_loop.c" depth="0">
      <expressions>
        <expression name="-1">-1</expression>
        <expression name="k">0</expression>
        <expression name="argv">0x7ffffffffffe688</expression>
        <expression name="argc">1</expression>
      </expressions>
    </function>
  </trace>
</epoch>
</history>
</crash>
```

Listing A.2: A backtracking enabled Monkey XML bug report for a program with a division by zero bug

Bibliography

- [1] Ieee standard classification for software anomalies. *IEEE Std 1044-2009 (Revision of IEEE Std 1044-1993)*, pages C1 –15, 7 2010.
- [2] H. Agrawal, R.A. De Millo, and E.H. Spafford. An execution-backtracking approach to debugging. *Software, IEEE*, 8(3):21 –26, may 1991.
- [3] Hiralal Agrawal, Richard A. Demillo, and Eugene H. Spafford. Debugging with dynamic slicing and backtracking. *Software Practice and Experience*, 23:589–616, 1993.
- [4] Jacob Appelbaum. torproject.org. <https://www.torproject.org>, July 2012.
- [5] Victor Boctor. mantisbt.org. <http://www.mantisbt.org/>, July 2012.
- [6] Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley Professional, 1995.
- [7] P.P.-S. Chen and J. Akoka. Optimal design of distributed information systems. *Computers, IEEE Transactions on*, C-29(12):1068 –1080, dec. 1980.
- [8] Mozilla Foundation. bugzilla.org. <http://bugzilla.org>, July 2012.
- [9] The Eclipse Foundation. eclipse.org. <http://eclipse.org>, July 2012.
- [10] E. Gamma. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Addison-Wesley, 1995.
- [11] Christian Grothoff. Gnutnet - gnu framework for secure peer-to-peer networking. <https://gnunet.org/>, January 2012.
- [12] L. Hiew. Assisted detection of duplicate bug reports, 2006.
- [13] IBM. Ibm. <http://www.ibm.com/us/en/>, April 2012.
- [14] IBM. Ibm - db2- database software. <http://www-01.ibm.com/software/data/db2/>, April 2012.
- [15] IMS. Ibm - ims - information management system. <http://www-01.ibm.com/software/data/ims/>, April 2012.
- [16] Java.net. Javacc home. <http://javacc.java.net/>, May 2012.
- [17] Zhang Junhua, Zhu Xiaodong, and Huang Zhiqiu. Parallel programming patterns with granular computing. In *Computer Application and System Modeling (ICCA SM), 2010 International Conference on*, volume 7, pages V7–300 –V7–302, oct. 2010.

- [18] Yann-Hang Lee, Young Wn Song, R. Girmé, S. Zaveri, and Yan Chen. Replay debugging for multi-threaded embedded software. In *Embedded and Ubiquitous Computing (EUC), 2010 IEEE/IFIP 8th International Conference on*, pages 15–22, dec. 2010.
- [19] libmigdb. Gdb machine interface library. <http://sourceforge.net/projects/libmigdb/>, July 2012.
- [20] N.K. Nagwani and P. Singh. Weight similarity measurement model based, object oriented approach for bug databases mining to detect similar and duplicate bugs. In *Proceedings of the International Conference on Advances in Computing, Communication and Control*, pages 202–207. ACM, 2009.
- [21] Jens Palsberg. Jtb: The java tree builder homepage. <http://compilers.cs.ucla.edu/jtb/>, May 2012.
- [22] Julian Seward. Valgrind. <http://www.valgrind.org>, January 2012.
- [23] TMaté Software. Sqljet. <http://sqljet.com/>, May 2012.
- [24] sqlite.org. Sqlite home page. <http://www.sqlite.org/>, January 2012.
- [25] Richard Stallman. Gdb - the gnu project debugger. <http://www.gnu.org/software/gdb/>, January 2012.
- [26] M. Sullivan and R. Chillarege. A comparison of software defects in database management systems and operating systems. In *Fault-Tolerant Computing, 1992. FTCS-22. Digest of Papers., Twenty-Second International Symposium on*, pages 475–484, jul 1992.
- [27] ubuntu.com. Apport - ubuntu wiki. <https://wiki.ubuntu.com/Apport>, January 2012.
- [28] Mark Weiser. Program slicing. *IEEE Trans. Software Eng.*, 10(4):352–357, 1984.